

Algorithm Templates — Master Reference

High-level templates, core algorithms, and quick-reference tables for every pattern. Detailed per-problem walkthroughs and side-by-side comparisons live in each category's own file (e.g. `dynamic_programming.md`, `graph.md`).

Common Java API Cheat-Sheet

The APIs used over and over across these templates. Grouped by what you reach for.

Initialize the core structures

```
Map<Integer, Integer> map = new HashMap<>(); // key → value
Map<Integer, List<Integer>> adj = new HashMap<>(); // adjacency / buckets
Set<Integer> set = new HashSet<>(); // membership
List<Integer> list = new ArrayList<>(); // dynamic array

Deque<Integer> stack = new ArrayDeque<>(); // STACK: push / pop / peek
Deque<Integer> queue = new ArrayDeque<>(); // QUEUE: offer / poll / peek
// (ArrayDeque beats java.util.Stack / LinkedList)

PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // min-heap (default)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // by field, min-first

TreeMap<Integer, Integer> tmap = new TreeMap<>(); // sorted keys: floor/ceiling/first/last
TreeSet<Integer> tset = new TreeSet<>(); // sorted set: floor/ceiling/higher/lower
LinkedHashSet<Integer> lset = new LinkedHashSet<>(); // insertion-ordered set (LFU buckets)
Random rand = new Random(); // rand.nextInt(n) → 0..n-1 (RandomizedSet)

int[] arr = new int[n]; // defaults to 0
boolean[] seen = new boolean[n]; // defaults to false
int[][] grid = new int[m][n]; // 2D, all 0
int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}}; // UP, DOWN, LEFT, RIGHT
```

Map / Set / List operations

```
// — Map —
map.put(k, v);
map.getOrDefault(k, 0); // safe read with default
map.merge(k, 1, Integer::sum); // ← counting frequency (most common)
map.computeIfAbsent(k, x -> new ArrayList<>().add(v)); // ← build adjacency / buckets
map.putIfAbsent(k, v); // ← write only first occurrence (keep earliest index)
map.containsKey(k); map.remove(k);
for (Map.Entry<Integer,Integer> e : map.entrySet()) { e.getKey(); e.getValue(); e.setValue(v); }
map.keySet(); map.values();

// — Set —
set.add(x); set.contains(x); set.remove(x);

// — List —
list.add(x); list.get(i); list.set(i, x);
list.remove(list.size() - 1); // ← pop in backtracking
list.size(); list.isEmpty(); list.addAll(other);
Collections.sort(list); // ascending
list.sort((a, b) -> b - a); // custom / descending
Collections.reverse(list); // ← reverse path after building it backwards
```

Stack / Queue / Heap (all via the same two ops)

```
// Deque as STACK (LIFO) // Deque as QUEUE (FIFO)
stack.push(x); queue.offer(x);
int top = stack.pop(); int head = queue.poll();
int look = stack.peek(); int look = queue.peek();
stack.isEmpty(); queue.isEmpty();

// Deque BOTH ENDS — needed for the MONOTONE DEQUE (sliding-window max/min)
// and for addFirst-style level building (zigzag BFS).
deque.offerLast(x); deque.pollLast(); deque.peekLast(); // back (push/pop/peek tail)
deque.offerFirst(x); deque.pollFirst(); deque.peekFirst(); // front (push/pop/peek head)
```

```

deque.addLast(x);      deque.addFirst(x);          // same as offer*, throw if capacity-bound

// PriorityQueue (heap)
pq.offer(x); int best = pq.poll(); int look = pq.peek(); pq.size();
pq.remove(x); // ← O(n) removal of an arbitrary element (lazy-deletion alternative in #480)

```

Integer / Long limits & overflow-safe idioms

```

Integer.MAX_VALUE; // 2147483647 (2^31 - 1) - sentinel for "min so far"
Integer.MIN_VALUE; // -2147483648 (-2^31) - sentinel for "max so far"
Long.MAX_VALUE; // when sums can overflow int, accumulate in long
int mid = i + (j - i) / 2; // ← avoid (i + j) overflow in binary search
Integer.compare(a, b); // ← overflow-safe comparator; use instead of (a - b)
(a, b) -> Integer.compare(a[1], b[1]); // ← safe PriorityQueue/Arrays.sort comparator
Double.compare(a, b); // ← comparator for double[] heaps (probability, median)

```

Char ↔ int ↔ String conversions

```

ch = 'a'; // 'a'..'z' → 0..25 (lowercase bucket index)
ch = '0'; // '0'..'9' → 0..9 (digit value)
(char) ('a' + i); // 0..25 → 'a'..'z'
num = num * 10 + (ch - '0'); // ← build a number digit by digit

Integer.parseInt("123"); // String → int (throws on bad input)
Integer.valueOf(123); // int → Integer (boxed)
String.valueOf(123); // int/char/etc → String
Character.getNumericValue('7'); // char → 7

// char classification
Character.isDigit(ch); Character.isLetter(ch); Character.isLetterOrDigit(ch);
Character.isWhitespace(ch); Character.toLowerCase(ch); Character.toUpperCase(ch);

```

String & StringBuilder

```

s.length(); s.charAt(i); s.substring(i, j); // [i, j)
s.toCharArray(); s.split(" "); s.equals(t); s.compareTo(t);
s.indexOf("ab"); s.isEmpty();
s.startsWith("ab"); s.endsWith("z"); s.contains("x"); // prefix / suffix / substring tests
s.toLowerCase(); s.toUpperCase(); // case-normalize (e.g. case-insensitive compare)
s.trim(); s.replace('a', 'b'); s.repeat(n); // strip ends / swap chars / repeat

StringBuilder sb = new StringBuilder();
sb.append(x); sb.reverse(); sb.toString();
sb.charAt(i); sb.setCharAt(i, c); sb.deleteCharAt(i); sb.length();
String.join(",", list); // list → "a,b,c"

```

Arrays utilities

```

Arrays.sort(arr); // ascending in place
Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // 2D by first field
Arrays.fill(dp, Integer.MAX_VALUE); // seed a dp/dist array
Arrays.equals(window, need); // ← element-wise array compare (anagram check)
Arrays.copyOf(arr, len); Arrays.copyOfRange(arr, i, j);
System.arraycopy(src, srcPos, dst, dstPos, len); // ← fast block copy (merge step)
Arrays.asList(1, 2, 3); // fixed-size List view
int sum = Arrays.stream(arr).sum();
int max = Arrays.stream(arr).max().getAsInt();

// List<Integer> ↔ int[]
int[] a = list.stream().mapToInt(Integer::intValue).toArray();
List<Integer> l = Arrays.stream(a).boxed().collect(Collectors.toList());

```

Math & bit operations

```
Math.max(a, b); Math.min(a, b); Math.abs(x);
Math.pow(a, b); Math.sqrt(x); Math.ceil(x); Math.floor(x);

n & (n - 1);           // clear lowest set bit
n & (-n);              // isolate lowest set bit
1 << i;                // bit i set (use 1L << i for i ≥ 31)
(n >> i) & 1;          // read bit i
(n & 1) == 0;          // ← even (last bit 0)
(n & 1) == 1;          // ← odd (last bit 1)
n >> 1;               // ← divide by 2 (drop last bit); n << 1 = multiply by 2
Integer.bitCount(n);   // # of set bits
Integer.toBinaryString(n);
```

Sorted-structure navigation (TreeMap / TreeSet)

```
tmap.firstKey(); tmap.lastKey();
tmap.floorKey(x); // largest key ≤ x      tmap.ceilingKey(x); // smallest key ≥ x
tmap.lowerKey(x); // largest key < x      tmap.higherKey(x); // smallest key > x
tset.floor(x); tset.ceiling(x); tset.lower(x); tset.higher(x); tset.first(); tset.last();
```

Two Pointers **two_pointers.md**

Two patterns — pick based on whether pointers converge or march together.

Quick Reference Table

#	Name	Description	Pattern	i init	j init	Loop	Duplicates
167	Two Sum II	Two numbers in sorted array summing to target	Opposite	0	n-1	i < j	No
15	3Sum	All unique triplets summing to 0	Opposite + fix	a+1	n-1	i < j	Skip after hit
18	4Sum	All unique quadruplets summing to target	Opposite + 2 fix	b+1	n-1	i < j	Skip after hit
11	Container With Most Water	Max water between two vertical lines	Opposite	0	n-1	i < j	No
42	Trapping Rain Water	Total trapped water between elevation bars	Opposite	0	n-1	i < j	No
27	Remove Element	Remove all val in-place, return new length	Same dir	0 (write)	0 (read)	j < n	No
26	Remove Duplicates	Each element appears at most once in-place	Same dir	0 (write)	0 (read)	j < n	Compare nums[i-1]
80	Remove Duplicates II	Each element appears at most twice in-place	Same dir	0 (write)	0 (read)	j < n	Compare nums[i-2]
283	Move Zeroes	Move zeroes to end, preserve relative order	Same dir	0 (write)	0 (read)	j < n	No
75	Sort Colors	Sort 0s, 1s, 2s in-place (Dutch Flag)	3-pointer	0 (low)	n-1 (high)	k <= j	No

Pattern 1 — Opposite Two Pointers

```
// MENTAL MODEL: two ends close in on the answer; each comparison rules out one whole end of the range.
// WHY sorted: moving inward only safely eliminates half the space if the array is sorted
int i = 0, j = n - 1;
while (i < j) {
    if (condition == target) {
        /* record */ i++; j--;
    } else if (tooSmall) {
        i++;
    } else {
        j--;
    }
}
```

```
        j--;  
    }  
}
```

Sort first. `i` and `j` converge inward. Used when the array is sorted and you can eliminate half the search space each step.

Pattern 2 — Same Direction (Write Pointer)

```
// MENTAL MODEL: a fast reader scans everything; a slow writer trails behind and only copies the keepers.  
// WHEN: filter/compact an array in place, no extra buffer  
int i = 0;           // i = write position  
for (int j = 0; j < n; j++) {      // j = read position  
    if (keepCondition(nums[j]))  
        nums[i++] = nums[j];  
}  
return i;
```

`i` marks where the next valid element goes. `j` scans every element. Only write when the element passes the keep condition.

The generalised duplicate-skip pattern for "at most k duplicates":

```
int i = 0;  
for (int j = 0; j < n; j++)  
    if (i < k || nums[j] != nums[i - k]) // compare write head to k slots back  
        nums[i++] = nums[j];  
return i;
```

- $k = 1 \rightarrow$ #26 (no duplicates)
 - $k = 2 \rightarrow$ #80 (at most 2)
-

Pattern 3 — Dutch Flag (3 Pointers)

```
// MENTAL MODEL: one scan partitions into three buckets (low / mid / high) using two boundary pointers.  
// WHEN: 3-way partition - sort 0/1/2, segregate by a pivot category  
int i = 0, k = 0, j = n - 1;  
while (k <= j) {  
    if (nums[k] == LO) {  
        swap(nums, i++, k++); // confirmed small  
    } else if (nums[k] == MID) {  
        k++; // confirmed mid, skip  
    } else {  
        swap(nums, k, j--); // large - don't advance k  
    }  
}
```

Three regions: $[0, i)$ = confirmed small, $[i, k)$ = confirmed mid, (j, n) = confirmed large. `k` is the frontier. Do **not** increment `k` after swapping with `j` — the swapped element is unknown.

Pattern Comparison

Pattern	Pointers move	Sort needed	Use when
Opposite	i right, j left, converge	Yes (usually)	Sum/target problems on sorted array
Same direction	j scans, i writes	No	Filter / compact array in-place
Dutch Flag	k scans, i low boundary, j high boundary	No	3-way partition ($< / = / >$)

Duplicate-skip Cheat Sheet (for Opposite pointers after a hit)

```
// After recording a result at (i, j):
i++; j--;
while (i < j && nums[i] == nums[i - 1]) i++; // skip duplicate i
while (i < j && nums[j] == nums[j + 1]) j--; // skip duplicate j
```

Always check `i < j` inside the skip loop to avoid crossing.

Sliding Window `sliding_window.md`

Three variants — pick by what the problem asks for.
`i` = left boundary (inclusive), `j` = right boundary (loop variable, inclusive).
Window = `[i, j]` , size = `j - i + 1` .

The Three Templates

```
// FIXED window of size k - shrink exactly once when full
// MENTAL MODEL: a constant-width window slides one step at a time; add the new cell, drop the old one.
// WHEN: "subarray/substring of size k"
int i = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    if (j - i + 1 == k) {
        // 2. record (window is exactly size k)
        // 3. remove nums[i] from state
        i++;
    }
}

// MAX variable window - find LONGEST valid window
// MENTAL MODEL: grow greedily; shrink only enough to restore validity, then the window is the best ending here.
// WHEN: "longest" + "at most k ..."
int i = 0, result = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window exceeds constraint */) { // shrink WHILE window exceeds constraint
        // remove nums[i] from state
        i++;
    }
    result = Math.max(result, j - i + 1); // record after window is valid
}

// MIN variable window - find SHORTEST valid window
// MENTAL MODEL: grow until valid, then shrink aggressively while still valid to find the tightest fit.
// WHEN: "shortest/minimum" + "sum >= target" / "contains all of t"
int i = 0, result = Integer.MAX_VALUE;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window VALID */) { // record then shrink WHILE window still valid
        result = Math.min(result, j - i + 1); // record while valid
        // remove nums[i] from state
        i++; // then try to shrink
    }
}
```

Quick Reference Table

#	Name	Description	Problem phrase trigger	Type	State	Shrink / Record condition
643	Max Average Subarray I	Max average of subarray of size k	"subarray of size k"	Fixed	sum	shrink when <code>j-i+1 == k</code>
567	Permutation in String	Does s2 contain any permutation of s1?	"substring of size len(s1)" / "permutation"	Fixed	freq + required count	shrink when <code>j-i+1 == len(s1)</code>

219	Contains Duplicate II	Any duplicate indices $\leq k$ apart	"within k indices"	Fixed	HashSet	shrink when $j-i+1 > k$
239	Sliding Window Maximum	Max element in every window of size k	"max of every window of size k"	Fixed	monotone deque	shrink when front out of $[i, j]$
1004	Max Consecutive Ones III	Longest subarray with at most k zeros	"longest" + "at most k zeros"	Max	zero count	shrink while zeros $> k$
3	Longest No-Repeat Substring	Longest substring without duplicates	"longest" + "no repeats"	Max	freq array	shrink while $freq[j] > 1$
424	Longest Char Replacement	Longest with at most k replacements	"longest" + "at most k replacements"	Max	freq + maxFreq	shrink while size - maxFreq $> k$
209	Min Size Subarray Sum	Shortest subarray with sum $\geq$ target	"shortest" + "sum $\geq$ target"	Min	sum	record + shrink while sum $\geq$ target
76	Minimum Window Substring	Shortest substring containing all of t	"shortest substring containing" all of t	Min (freq)	freq + required count	record + shrink while required == 0
159	Longest Substring with At Most 2 Distinct	Max length substring with ≤ 2 distinct chars	"longest" + "at most 2 distinct"	Max	freq map (size ≤ 2)	shrink while $freq.size() > 2$
340	Longest Substring with At Most K Distinct	Max length substring with $\leq k$ distinct chars	"longest" + "at most k distinct"	Max	freq map (size $\leq k$)	shrink while $freq.size() > k$
438	Find All Anagrams in a String	Find all start indices of p's anagrams in s	"all anagrams" / "all windows of size len(p)"	Fixed	freq compare <code>int[26]</code>	shrink when $j-i == len(p)$
1343	Number of Subarrays of Size K and Avg $\geq$ Threshold	Count fixed-size windows with average $\geq$ threshold	"subarrays of size k" + "average $\geq$ threshold"	Fixed	sum (compare sum $\geq k \cdot threshold$)	slide when window size == k

Frequency Map Trick (used by 567 and 76)

Both use a `need[]` array and a `required` counter to avoid scanning the map each step.

```
// Expanding j - add s.charAt(j):
if (need[s.charAt(j)]-- > 0) required--; // was needed → satisfied one more

// Shrinking i - remove s.charAt(i):
if (need[s.charAt(i)]++ >= 0) required++; // was 0 (used up) → now short one
i++;
```

`need[c]` is positive when still needed, zero or negative when excess.
`required` reaches 0 exactly when all characters of t are covered.

Choosing the Template

Question asks for	Template	i moves
Fixed-size window operation	Fixed	when $j-i+1 == k$
Longest / maximum window	Max variable	while invalid (while)
Shortest / minimum window	Min variable	while valid (while)

Window content	State to maintain
Sum of values	<code>int sum</code>
Unique characters / no duplicates	<code>int[] freq</code> (128 or 26) or <code>Set</code>
Character frequency match	<code>int[] need</code> + <code>int required</code> counter
Running maximum	<code>Deque<Integer></code> monotone decreasing indices

Prefix Sum + HashMap prefix_sum_hashmap.md

Core idea: `subarray sum [l, r] = prefix[r+1] - prefix[l]` .
To find a subarray with a target property, rephrase as: "has a prefix with value X been seen before?"
Store that X in a HashMap for O(1) lookup.

Core Template

```
// MENTAL MODEL: a subarray is the difference of two prefixes; remember prefixes seen so far and look up the complement.
// WHEN: "subarray sum equals/divisible by k", "longest subarray with sum X" - anything reducible to prefix[r]-prefix[l]
Map<Long, Integer> map = new HashMap<>();
map.put(0L, ???); // seed: empty prefix (sum=0) seen before index 0
long sum = 0;
// result = 0 (count) or Integer.MAX_VALUE (length sentinel) depending on problem

for (int i = 0; i < nums.length; i++) {
    sum += nums[i];
    long lookup = transform(sum); // what to look up (sum-k, sum%k, etc.)
    // use map.get(lookup) -> update result
    map.put(storeKey(sum), storeValue(i)); // what to store (sum or sum%k -> count or index)
}
```

What changes per problem:

Variable	Count problems (560, 974)	Length problems (325, 523)
Map value	count (how many times seen)	first index (earliest occurrence)
Seed value	map.put(0L, 1)	map.put(0L, -1)
On lookup hit	count += map.get(lookup)	maxLen = max(i - map.get(lookup))
Overwrite map?	Yes (merge count)	No (keep first index only)

Quick Reference Table

Read the two map-value modes first: **count** problems store `prefix sum -> how many times seen` and answer "how many"; **length/index** problems store `prefix sum -> earliest index seen` and answer "how long / where".
The two modes also seed differently:

- Seed `(0, 1)` = the empty prefix has been **seen once** before we start — needed for *counting* (a prefix that itself equals the target counts as one valid subarray).
- Seed `(0, -1)` = the empty prefix lives at a **virtual index -1** — needed for *length* (length from start = `i - (-1)` = `i + 1`).

#	Name	What you store in map	Seed (why)	Lookup key	Result calc
560	Subarray Sum Equals K	prefix sum -> count seen	(0L, 1) — empty prefix seen once, for counting	sum - k	count += map[sum-k]
325	Max Size Subarray Sum = k	prefix sum -> earliest index	(0L, -1) — empty prefix at virtual index -1, for length	sum - k	maxLen = max(i - map[sum-k])
974	Subarray Sums Divisible by K	prefix sum % k -> count seen	(0, 1) — empty prefix seen once, for counting	sum % k	count += map[sum%k]
523	Continuous Subarray Sum	prefix sum % k -> earliest index	(0, -1) — empty prefix at virtual index -1, for length	sum % k	i - map[sum%k] >= 2 -> true
437	Path Sum III	prefix sum -> count seen (on current path)	(0L, 1) — empty prefix seen once, for counting	sum - target	count += map[sum-target], backtrack on return
1	Two Sum	value -> index (raw value, not a running sum)	none — nothing seen before first element	target - nums[i]	[map[complement], i]

When to Use `long` vs `int`

- Use `long` for the prefix sum when values can be large (e.g., `nums[i]` up to $10^4 \times n$ up to $2 \times 10^4 \rightarrow$ sum up to 2×10^8 , safe as `int`, but $10^5 \times 10^5 = 10^{10}$ overflows).
- Use `int` for the remainder key (modulo result is always in `[0, k-1]`).

Why `0L` (not `0`) in the seed

When the map is `Map<Long, Integer>`, the seed **key** must be a `long` literal:

```
Map<Long, Integer> map = new HashMap<>();
map.put(0L, 1);      // ✓ long literal → boxes to Long (matches key type)
map.put(0, 1);       // ✗ compile error: int boxes to Integer, not Long
```

- The `L` suffix makes it a `long` literal so it autoboxes to `Long`, matching the key type (the keys are `long` because `sum` is `long` for the overflow guard above).
- **Subtle trap:** even numerically equal boxes of different types never compare equal — `Long.valueOf(0).equals(Integer.valueOf(0))` is `false`. Keeping every key `long / Long` avoids silent lookup misses.
- If `int` sums can't overflow, use `Map<Integer, Integer>` with plain `map.put(0, 1)` instead — simpler, no `L` needed.

Binary Search `binary_search.md`

Binary search appears in two forms: searching an **array index**, or searching an **answer value**.

Master Quick Reference

#	Name	Category	Template	i init	j init	Loop	Return
704	Binary Search	Array index	Closed	0	n-1	i <= j	-1
34	First & Last Position	Array index	Half-open	0	n	i < j	{lower, upper-1}
35	Search Insert Position	Array index	Half-open	0	n	i < j	i
33	Search in Rotated Array	Array index	Closed	0	n-1	i <= j	-1
162	Find Peak Element	Array index	Half-open*	0	n-1	i < j	i
875	Koko Eating Bananas	Answer space	MINIMIZE	1	max(piles)	i < j	i
1011	Ship Within D Days	Answer space	MINIMIZE	max(w)	sum(w)	i < j	i
410	Split Array Largest Sum	Answer space	MINIMIZE	max(nums)	sum(nums)	i < j	i
1283	Smallest Divisor	Answer space	MINIMIZE	1	max(nums)	i < j	i
1231	Divide Chocolate	Answer space	MAXIMIZE	min(s)	sum(s)/(k+1)	i < j	i

* #162 uses `j = n-1` (not `n`) — loop reads `arr[k+1]`, out of bounds if `j = n`.

All Four Templates

```
// 1. CLOSED [i, j] — exact match, return on hit
// MENTAL MODEL: target is a specific value sitting at some index; shrink a fully-closed range until you land on it.
// WHEN: "find this exact value in a sorted array"
int i = 0, j = n - 1;
while (i <= j) {
    int k = i + (j - i) / 2;
    if (arr[k] == target) {
        return k;
    } else if (arr[k] < target) {
        i = k + 1;
    } else {
        j = k - 1;
    }
}
```



```

    }
}
return -1;

// 2. HALF-OPEN [i, j) – boundary search, answer falls out of i
// MENTAL MODEL: the predicate is false...false TRUE...true; find the FIRST position that satisfies the condition.
// WHEN: "first/last position", "insert position", "first index where condition holds"
int i = 0, j = n;          // j = n (or n-1 for #162)
while (i < j) {
    int k = i + (j - i) / 2;
    if (arr[k] < target) {
        i = k + 1;        // lowerBound: <
    } else {
        j = k;            // upperBound: swap < for <=
    }
}
return i;

// 3. ANSWER SPACE MINIMIZE – smallest k where condition(k) is true
// MENTAL MODEL: condition goes false→TRUE as k grows (bigger k = more feasible); find the smallest k that flips it true.
// WHEN: "minimum X such that feasible(X)" – min speed, min capacity, smallest divisor
int i = minPossible, j = maxPossible;
while (i < j) {
    int k = i + (j - i) / 2;          // floor mid
    if (condition(k)) {
        j = k;
    } else {
        i = k + 1;
    }
}
return i;

// 4. ANSWER SPACE MAXIMIZE – largest k where condition(k) is true
// MENTAL MODEL: condition goes TRUE→false as k grows (bigger k = less feasible); find the largest k still true.
// WHEN: "maximum X such that feasible(X)" – max min-piece, max guarantee
int i = minPossible, j = maxPossible;
while (i < j) {
    int k = i + (j - i + 1) / 2;      // ceiling mid ← avoids infinite loop
    if (condition(k)) {
        i = k;
    } else {
        j = k - 1;
    }
}
return i;

```

How to Choose

Signal	Template
Sorted array, find exact value	Closed — return on <code>arr[k] == target</code>
First/last position, insert position	Half-open lowerBound / upperBound
Rotated array or slope search	Closed — pick sorted half each step
"minimum X such that feasible(X)"	Answer space MINIMIZE — floor mid
"maximum X such that feasible(X)"	Answer space MAXIMIZE — ceiling mid

Detail Table

#	Name	Description	Interval	On <code>arr[k]==target</code>	Shrink left	Shrink right	Return
704	Binary Search	Find index of target; -1 if absent	Closed	return k	i = k+1	j = k-1	-1
34	First & Last Position	First and last index of target	Half-open	—	i = k+1	j = k	{lower, upper-1}
35	Search Insert Position	Index where target is or would be inserted	Half-open	—	i = k+1	j = k	i
33	Search in Rotated Array	Find target after unknown-pivot rotation	Closed	return k	i = k+1	j = k-1	-1
162	Find Peak Element	Any local peak index	Half-open*	—	i = k+1	j = k	i

Search on the **answer value** itself, not an array index. All use half-open `[i, j)` with `while (i < j)`.

Detail Table

#	Name	Description	Variant	i init	j init	Mid	Condition true	Condition false	Helper checks
875	Koko Eating Bananas	Min speed to finish all piles in h hours	MINIMIZE	1	$\max(\text{piles})$	floor	$j = k$	$i = k+1$	$\text{hours} \leq h$
1011	Ship Within D Days	Min capacity to ship all packages in d days	MINIMIZE	$\max(w)$	$\text{sum}(w)$	floor	$j = k$	$i = k+1$	$\text{days} \leq d$
410	Split Array Largest Sum	Min largest sum splitting array into m parts	MINIMIZE	$\max(nums)$	$\text{sum}(nums)$	floor	$j = k$	$i = k+1$	$\text{parts} \leq m$
1283	Smallest Divisor	Smallest divisor so $\text{ceil-div-sum} \leq \text{threshold}$	MINIMIZE	1	$\max(nums)$	floor	$j = k$	$i = k+1$	$\text{divSum} \leq \text{threshold}$
1231	Divide Chocolate	Max min-sweetness cutting into k+1 pieces	MAXIMIZE	$\min(s)$	$\text{sum}(s)/(k+1)$	ceiling	$i = k$	$j = k-1$	$\text{pieces} \geq k+1$

875 / 1011 / 410 / 1283 → identical structure, differ only in search bounds and condition helper.

1231 → only one that flips (MAXIMIZE): ceiling mid, `i = k` on success.

Sorting `sorting.md`

All recursive sorting uses **half-open intervals** `[start, end)` throughout.

Inside merge and partition: `i`, `j`, `k` as scan/write pointers (consistent with binary search convention).

Quick Reference Table

#	Name	Description	Algorithm	Time	Space	Key
912	Sort an Array	Sort integer array	Merge Sort	$O(n \log n)$	$O(n)$	Stable; template base
912	Sort an Array	Sort integer array	Quick Sort	$O(n \log n)$ avg	$O(\log n)$	In-place; 3-way Dutch flag partition
215	Kth Largest Element	Find k-th largest without full sort	Quick Select	$O(n)$ avg	$O(\log n)$	Recurse ONE side; skip equal-pivot range
973	K Closest Points	K points closest to origin	Quick Select	$O(n)$ avg	$O(\log n)$	Same as 215 with custom dist comparator
148	Sort List	Sort a linked list	Merge Sort	$O(n \log n)$	$O(\log n)$	Slow/fast to find mid; split then merge
164	Maximum Gap	Max gap between successive elements in sorted order	Bucket sort: n+1 buckets sized $(\text{max-min})/n$; gap spans bucket boundaries	Pigeonhole buckets: max gap $\geq$ bucket size, so compare across bucket min/max	$O(n)$	$O(n)$
179	Largest Number	Form largest number from integers	Custom Sort	$O(n \log n)$	$O(n)$	Compare $b+a$ vs $a+b$ as strings
315	Count of Smaller After Self	Count smaller elements to the right	Merge Sort	$O(n \log n)$	$O(n)$	Track original indices; count during merge
56	Merge Intervals	Merge all overlapping intervals	Sort + Scan	$O(n \log n)$	$O(n)$	Sort by start; greedily extend end
88	Merge Sorted Array	Merge <code>nums2</code> into <code>nums1</code> in-place; both sorted	Three pointers from END: $i=m-1$, $j=n-1$, $k=m+n-1$; fill backwards	Fill backwards: avoid overwriting unread elements in <code>nums1</code>	$O(m+n)$	$O(1)$
327	Count of Range Sum	Count subarray sums within <code>[lower, upper]</code>	Merge sort on prefix sums; during merge count valid pairs across halves	Count during merge: maintain two sliding pointers <code>j,k</code> into right half while merging left half	$O(n \log n)$	$O(n)$

Template 1 — Merge Sort

```
// MENTAL MODEL: split in half until trivially sorted, then merge two sorted halves into one.
// WHEN: need stable sort, sort a linked list, or count cross-pairs (inversions) during the merge.
// Entry point
public void mergeSort(int[] nums) {
    mergeSort(nums, 0, nums.length, new int[nums.length]);
}

// [start, end) half-open
private void mergeSort(int[] nums, int start, int end, int[] temp) {
    if (end - start <= 1) return; // base case: end - start <= 1 → 0 or 1 elements, already sorted
    int mid = start + (end - start) / 2;
    mergeSort(nums, start, mid, temp); // sort [start, mid)
    mergeSort(nums, mid, end, temp); // sort [mid, end)
    merge(nums, start, mid, end, temp);
}

// i scans left [start,mid), j scans right [mid,end), k writes to temp
private void merge(int[] nums, int start, int mid, int end, int[] temp) {
    int i = start, j = mid, k = start;
    while (i < mid || j < end) {
        if (j >= end || (i < mid && nums[i] <= nums[j])) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }
    for (i = start; i < end; i++) nums[i] = temp[i];
}
```

Template 2 — Quick Sort (3-way Dutch Flag Partition)

```
// MENTAL MODEL: pick a pivot, partition into <pivot | ==pivot | >pivot, recurse on the two ends only.
// WHEN: in-place sort with O(log n) stack; the ==pivot middle is already in final position.
// Entry point — no extra array needed (in-place)
public void quickSort(int[] nums) {
    quickSort(nums, 0, nums.length);
}

// [start, end) half-open
private void quickSort(int[] nums, int start, int end) {
    if (end - start <= 1) return;
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    // Invariant: [start,i) < pivot | [i,k) == pivot | [k,j) unknown | (j,end) > pivot
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++, i++);
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    // After: [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
    quickSort(nums, start, i); // recurse on < region
    quickSort(nums, k, end); // recurse on > region
}

private void swap(int[] nums, int i, int j) {
    int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
}
```

Template 3 — Quick Select (partial sort — k-th element)

Extends quick sort: after partition, only recurse into the side that contains **target** index.

```
// MENTAL MODEL: same partition as quick sort, but recurse into only the one side holding target → O(n) avg.
// WHEN: "k-th largest/smallest", "k closest/most frequent" — you need a position, not a full sort.
// Returns value at 0-indexed position target after sorting
```

```
private int quickSelect(int[] nums, int start, int end, int target) {
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++, i++);
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    // [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
    if (target < i) {
        return quickSelect(nums, start, i, target);
    } else if (target < k) {
        return pivot; // target lands in == region
    } else {
        return quickSelect(nums, k, end, target);
    }
}
```

Template 4 — Counting Sort

$O(n + \text{range})$. Use when values are bounded integers.

```
// MENTAL MODEL: tally how many times each value occurs, then emit values in order — no comparisons.
// WHEN: integers in a small bounded range; beats  $O(n \log n)$  when range =  $O(n)$ .
public void countingSort(int[] arr) {
    if (arr.length == 0) return;
    int min = Arrays.stream(arr).min().getAsInt();
    int max = Arrays.stream(arr).max().getAsInt();
    int[] buckets = new int[max - min + 1];
    for (int x : arr) buckets[x - min]++;
    int i = 0;
    for (int v = 0; v < buckets.length; v++)
        while (buckets[v]-- > 0) arr[i++] = v + min;
}
```

Algorithm Chooser

Signal	Algorithm
"sort array", stable, $O(n)$ space OK	Merge Sort
"sort array", in-place, $O(\log n)$ space	Quick Sort
"k-th largest/smallest"	Quick Select — $O(n)$ avg
"k closest / k most frequent"	Quick Select with custom comparator
"sort linked list"	Merge Sort (quick sort needs random access)
"count inversions / smaller after self"	Merge Sort — count during merge step
"arrange to maximize/compare"	Custom Sort comparator
"merge overlapping intervals"	Sort by start + linear scan
integers in bounded range	Counting Sort — $O(n + \text{range})$

Half-open Interval Cheat Sheet

```
mergeSort(nums, start, end) → sorts [start, end)
mid = start + (end-start)/2
left:  [start, mid)
right: [mid, end)
base:  end - start <= 1

quickSort(nums, start, end) → sorts [start, end)
after partition: [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
recurse: (start, i) and (k, end)
```

Linked List linked_list.md

Four patterns cover almost every linked list problem.

Consistent naming throughout: sentinel , previous , current , next , slow , fast .

Quick Reference Table

#	Name	Description	Pattern	Sentinel	Key pointers
203	Remove Linked List Elements	Remove all nodes with value == val	Delete/Filter	Yes	previous, current
83	Remove Duplicates I	Keep one copy of each duplicate value	Delete/Filter	Yes	previous, current
82	Remove Duplicates II	Remove ALL nodes that have any duplicate	Delete/Filter	Yes	previous, current
19	Remove Nth From End	Remove nth node from end of list	Delete + fast/slow	Yes	fast, slow
206	Reverse Linked List	Reverse entire list	Reversal	No	previous, current
92	Reverse Linked List II	Reverse nodes from position left to right	Reversal (partial)	Yes	previous, current
25	Reverse Nodes in k-Group	Reverse every k nodes	Reversal (grouped)	Yes	previous, current
876	Middle of Linked List	Return the middle node	Fast / Slow	No	slow, fast
141	Linked List Cycle	Detect if cycle exists	Fast / Slow	No	slow, fast
142	Linked List Cycle II	Return cycle entry node	Fast / Slow	No	slow, fast
21	Merge Two Sorted Lists	Merge two sorted lists into one	Merge	Yes	current
23	Merge K Sorted Lists	Merge k sorted lists using min-heap	Merge + heap	Yes	current
143	Reorder List	L0→Ln→L1→Ln-1→... reorder in-place	Find mid + Reverse + Merge	No	slow, fast, previous, current
328	Odd Even Linked List	Group odd-index nodes then even-index nodes	Regroup	No	odd, even
2	Add Two Numbers	Add two numbers represented as reversed linked lists	Sentinel + carry loop; simultaneously advance both lists	Carry variable: maintain carry = sum / 10 across nodes	O(max(m,n))
88	Merge Sorted Array	Merge nums2 into nums1 in-place	Three-pointer from the END; fill backwards to avoid overwriting	Fill backwards: start i=m-1, j=n-1, k=m+n-1 to avoid shifting	O(m+n)
160	Intersection of Two Linked Lists	Find the node where two lists intersect	Two pointers: when one reaches end, redirect to other list's head	Redirect on null: a = (a==null)? headB:a.next; both travel m+n total	O(m+n)
234	Palindrome Linked List	Check if a linked list is a palindrome	Fast/slow to find middle → reverse second half → compare	Composite: fast/slow + reversal + comparison	O(n)

When to Use — Signal → Pattern

Signal in the prompt	Pattern
"remove / delete / skip" nodes by value or duplicate	Delete / Filter (sentinel + previous)
"reverse" the list, a sub-range, or in k-groups	Reversal
"middle node", "cycle", "nth node from the end"	Fast / Slow
"merge two sorted lists" (or k lists)	Merge (sentinel + heap for k)
chained steps ("reorder", "palindrome", "sort")	Composite (combine the above)

All Four Templates

```
// 1. DELETE / FILTER - sentinel guards head removal; previous tracks last kept node
// MENTAL MODEL: previous always points at the last node you decided to keep, so re-wiring it skips anything unwanted.
// WHEN: "remove / delete / skip nodes by value or duplicate"
ListNode sentinel = new ListNode(0);
sentinel.next = head;
ListNode previous = sentinel, current = head;
while (current != null) {
    if (shouldDelete(current)) {
        previous.next = current.next;    // skip node
    } else {
        previous = current;              // keep node, advance previous
    }
    current = current.next;
}
return sentinel.next;

// 2. REVERSAL - re-wire one node at a time
// MENTAL MODEL: flip each arrow to point backward; previous is the growing reversed list trailing behind current.
// WHEN: "reverse the list (whole, partial, or in k-groups)"
ListNode previous = null, current = head;
while (current != null) {
    ListNode next = current.next;    // save before overwrite
    current.next = previous;         // reverse pointer
    previous = current;              // advance
    current = next;
}
return previous;    // new head

// 3. FAST / SLOW - two pointers at different speeds
// MENTAL MODEL: fast covers twice the distance, so when it hits the end slow is exactly halfway; in a loop they must collide.
// WHEN: "middle node", "detect/locate cycle", "nth from end"
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
// slow is at middle (or cycle detection: slow == fast -> cycle found)

// 4. MERGE - sentinel collects nodes from two lists in order
// MENTAL MODEL: repeatedly splice the smaller head onto a growing result tail, like a zipper.
// WHEN: "merge two (or k) sorted lists"
ListNode sentinel = new ListNode(0);
ListNode current = sentinel;
while (a != null && b != null) {
    if (a.val <= b.val) {
        current.next = a;
        a = a.next;
    } else {
        current.next = b;
        b = b.next;
    }
    current = current.next;
}
current.next = (a != null) ? a : b;
return sentinel.next;
```

`previous` stays on the last **kept** node. When deleting, re-wire `previous.next` to skip `current`. When keeping, advance `previous` to `current`. Always advance `current`.

`fast` moves 2x per step. When `fast` reaches the end, `slow` is at the middle. When `slow == fast` after the start, a cycle is detected.

`sentinel` collects the merged result. `current` is the write pointer.

Problems that chain multiple patterns together.

Pattern Summary

Pattern	Sentinel	Naming	When to use
Delete / Filter	Yes	previous, current	Remove or skip nodes by condition

Reversal	No (Yes for partial)	previous, current, next	Re-wire pointer direction
Fast / Slow	No	slow, fast	Middle, cycle, nth from end
Merge	Yes	current	Combine two or more lists
Composite	Both	Mix of above	Reorder, sort, rearrange

Sentinel Cheat Sheet

SENTINEL: dummy node before head so head removal needs no special-casing; use it whenever the head itself might be removed/replaced.

```
// Always use sentinel when the head itself might be removed or replaced:
ListNode sentinel = new ListNode(0);
sentinel.next = head;
// ... operations ...
return sentinel.next; // head may have changed; sentinel.next is always correct
```

String string.md

Core idioms first, then problems grouped by pattern.
For sliding window string problems (#3, #76, #567, #424), see sliding_window.md .

Quick Reference Table

#	Name	Pattern	Key technique	Time / Space
242	Valid Anagram	Char count	count++ then --	O(n) / O(1)
383	Ransom Note	Char count	fill from magazine, drain from note	O(n) / O(1)
387	First Unique Character	Char count	scan for count == 1	O(n) / O(1)
49	Group Anagrams	Freq hash key	26-bucket key or sorted key	O(n-k) / O(n)
451	Sort Characters by Frequency	Bucket sort	count[], bucket by freq, iterate desc	O(n) / O(n)
5	Longest Palindromic Substring	Expand from center	expand(i,i) odd + expand(i,i+1) even	O(n²) / O(1)
647	Palindromic Substrings	Expand from center	same expand, count instead of max length	O(n²) / O(1)
8	String to Integer (atoi)	Parsing	skip space → sign → digits → overflow	O(n) / O(1)
14	Longest Common Prefix	Vertical scan	shrink prefix until all strings match	O(n-k) / O(1)
443	String Compression	Write pointer	read run, write char+count in-place	O(n) / O(1)
1047	Remove Adjacent Duplicates	Stack	push; pop if top == current	O(n) / O(n)

Core Idioms

```
// MENTAL MODEL: a lowercase string is a length-26 frequency vector; most string problems are vector ops on it.
// WHEN: "anagram", "char count", "group by letters", "first unique", "sort by frequency".
// 1. CHAR COUNT – lowercase letters
int[] count = new int[26];
// WHY ch - 'a': ASCII 'a'=97, so 'a'→0, 'b'→1, ..., 'z'→25 – a clean 0–25 bucket index into count[].
for (char ch : s.toCharArray()) count[ch - 'a']++; // ← ch - 'a' maps a→0, b→1, ..., z→25

// 2. CHAR COUNT – digits 0–9
int[] count = new int[10];
for (char ch : s.toCharArray()) count[ch - '0']++; // ← ch - '0' maps '0'→0, '9'→9

// 3. CHAR TO INTEGER – build number digit by digit
int num = 0;
for (int i = 0; i < s.length(); i++)
    num = num * 10 + (s.charAt(i) - '0'); // ← shift left × 10, add new digit

// 4. ANAGRAM HASH KEY – frequency-based (bucket sort style) O(k) vs sort O(k log k)
// WHY it works: anagrams share identical letter frequencies, so encoding the frequency vector
// ITSELF gives a unique key for the whole group – O(k) to build vs O(k log k) to sort the chars.
String hashKey(String s) {
```

```

int[] count = new int[26];
for (char ch : s.toCharArray()) count[ch - 'a']++;
StringBuilder builder = new StringBuilder();
for (int i = 0; i < 26; i++) {
    builder.append((char)('a' + i));           // ← letter as label
    builder.append(count[i]);                 // ← its count
}
return builder.toString();                   // e.g., "a2b0c1...z0"
}

// Alternative: sort the characters (simpler, slightly slower)
char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars);              // anagrams → same sorted key

// 5. BUCKET SORT – sort by frequency without comparison sort
int[] count = new int[128];                 // ASCII
for (char ch : s.toCharArray()) count[ch]++;
List<Character>[] buckets = new List[s.length() + 1]; // index = frequency
for (int i = 0; i < 128; i++) {
    if (count[i] > 0) {
        int freq = count[i];
        if (buckets[freq] == null) buckets[freq] = new ArrayList<>();
        buckets[freq].add((char) i);
    }
}
StringBuilder builder = new StringBuilder();
for (int freq = buckets.length - 1; freq > 0; freq--) { // ← descending frequency
    if (buckets[freq] == null) continue;
    for (char ch : buckets[freq])
        for (int i = 0; i < freq; i++) builder.append(ch);
}

// 6. EXPAND FROM CENTER – palindrome check in O(1) space
int expand(String s, int i, int j) {
    // i == j: odd length; i+1 == j: even length
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1; // ← length of palindrome found
}

```

Template

```

// Call for each center: odd-length palindromes (i == i) and even-length (i, i+1)
for (int i = 0; i < s.length(); i++) {
    // odd: single center at i
    // even: center between i and i+1
    processExpansion(s, i, i); // odd
    processExpansion(s, i, i + 1); // even
}

// Shared expand helper – returns length of palindrome
int expand(String s, int i, int j) {
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1;
}

```

String Idioms Cheat Sheet

```

// Char ↔ index
'a' + i           // int → char (i=0→'a', i=25→'z')
ch - 'a'          // char → index (a→0, z→25)
ch - '0'          // char → digit (0→0, 9→9)
Character.isDigit(ch)
Character.isLetter(ch)
Character.toLowerCase(ch)

// String operations
s.charAt(i)           // char at index
s.toCharArray()       // → char[]
new String(charArray) // char[] → String
String.valueOf(charArray) // char[] → String
s.substring(i, j)     // [i, j)
s.contains(sub)        // boolean
s.startsWith(prefix)  // boolean
s.split("\\s+")        // split on whitespace

```



```
// Building strings
StringBuilder builder = new StringBuilder();
builder.append(ch);        builder.append(str);
builder.deleteCharAt(i);
builder.reverse();
builder.toString();
String.join(", ", list)           // join with delimiter

// computeIfAbsent – key pattern for grouping
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
```

Pattern Chooser

Signal	Pattern
Are two strings anagrams?	Count array: fill from s, drain from t
Group strings by anagram	Frequency hash key (hashCode) or Arrays.sort key
Sort/rank by character frequency	Bucket sort: count[], then buckets[freq]
Longest/count palindromic substrings	Expand from center (i,i) odd + (i,i+1) even
Parse a number from string	num = num * 10 + (ch - '0')
Remove adjacent duplicate pairs	Stack: push, pop on match
Sliding window over characters	See sliding_window.md

Stack / Queue / Heap stack_queue.md

Four data structures — each with a canonical template and representative problems.

Quick Reference Table

#	Name	Structure	Pattern	Pop/poll condition	Return
20	Valid Parentheses	Stack	Bracket matching	pop on close bracket	boolean
155	Min Stack	Stack + aux stack	Mirror min alongside values	always pop both	int (min)
394	Decode String	Two stacks	Push on [, expand on]	pop on]	String
739	Daily Temperatures	Mono dec stack	Next greater temp	current > top	int[]
496	Next Greater Element I	Mono dec stack + Map	NGE across two arrays	current > top	int[]
84	Largest Rectangle	Mono inc stack	Next smaller bar → compute width	current < top	int (area)
42	Trapping Rain Water	Mono inc stack	Valley between bars	current > top	int (water)
239	Sliding Window Max	Mono dec deque	Window max at front	pollLast when current ≥ back	int[]
1046	Last Stone Weight	Max-heap	Greedily smash top 2	always poll 2	int
347	Top K Frequent	Min-heap size k	Evict smallest when size > k	size > k → poll	int[]
295	Find Median	Max-heap + Min-heap	Lower/upper halves	rebalance after each add	double
502	IPO	Sort + Max-heap	Unlock by capital, pick max profit	w >= required capital	int
767	Reorganize String	Max-heap	Interleave two most frequent chars	always poll 2	String
85	Maximal Rectangle	Largest rectangle containing only 1s in binary matrix	For each row build histogram heights → apply #84 largestRectangleArea	Row-by-row histogram: reduce 2D problem to repeated #84	O(m·n)
224	Basic Calculator	Evaluate expression with +, -, ()	Stack saves (result, sign) before each (; restore after)	Sign + parentheses stack: push (result,sign) on (, pop and combine on)	O(n)
227	Basic Calculator II	Evaluate expression with +, -, *, / (no parentheses)	Scan tokens; push negative/positive for +/-; multiply/divide top of stack for */÷	Operator-before-operand: apply *, / immediately; defer +, - via signed push	O(n)
373	Find K Pairs with Smallest Sums	Find k pairs (u,v) with smallest u+v, u from nums1, v from nums2	Min-heap seeded with (nums1[i], nums2[0]) for all i; expand j on pop	Expand j: each heap entry tracks (i, j); pop → push (i, j+1)	O(k log k)

378	Kth Smallest Element in Sorted Matrix	kth smallest in n×n matrix where rows and cols are sorted	Binary search on value range [min,max]; count elements ≤ mid from bottom-left	Binary search on value: count(mid) ≥ k → search lower; else search higher	O(n log(max-min))
503	Next Greater Element II	Next greater element in circular array	Monotone decreasing stack; iterate array twice (i % n)	Circular: loop 2n, use i % n; only push indices in first pass (i < n)	O(n)
621	Task Scheduler	Minimum intervals to finish tasks with cooldown n	Count frequencies; answer = max((maxFreq-1)*(n+1)+maxCount, tasks.length)	Greedy formula: no simulation needed, direct calculation	O(k) k=tasks.length
358	Rearrange String k Distance Apart	Rearrange so same chars are ≥k apart	Max-heap by frequency + cooldown queue of size k	Cooldown queue: hold used char until k slots pass, then re-add to heap	O(n log k)
480	Sliding Window Median	Median of each window of size k	Two heaps (maxHeap lower, minHeap upper) + lazy deletion via HashMap	Lazy deletion: defer removing out-of-window elements; rebalance by counts	O(n log k)
692	Top K Frequent Words	k most frequent words, lexicographic on ties	Min-heap of size k ordered by (freq asc, word desc)	Custom tie-break: same freq → larger word first so it's evicted	O(n log k)
772	Basic Calculator III	Evaluate expression with +,-,*,/ and parentheses	Recursive/stack: handle parentheses recursively, apply *,/ immediately	Full calculator: combine #224 parentheses + #227 operator precedence	O(n)

All Templates

```
// 1. STACK (LIFO) – use ArrayDeque, not Stack class
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);    // add to top
stack.pop();      // remove from top
stack.peek();     // view top, no remove

// 2A. MONOTONE DECREASING STACK – next GREATER element
// MENTAL MODEL: keep only candidates still waiting for a bigger neighbor; current resolves them all.
// WHEN: "next/previous greater element", "warmer/taller to the right"
// Invariant: stack values decrease from bottom to top
// Pop when current is GREATER than top → top found its next greater
Deque<Integer> stack = new ArrayDeque<>(); // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i])
        result[stack.pop()] = nums[i]; // nums[i] is next greater for popped index
    stack.push(i);
}
// Remaining in stack: no next greater element exists

// 2B. MONOTONE INCREASING STACK – next SMALLER element / span width
// MENTAL MODEL: each popped bar's reach extends until something shorter blocks it on both sides.
// WHEN: "largest rectangle", "trapped water", "span/width bounded by smaller elements"
// Invariant: stack values increase from bottom to top
// Pop when current is SMALLER than top → use popped index to compute width/span
Deque<Integer> stack = new ArrayDeque<>(); // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
        int height = nums[stack.pop()];
        int width = stack.isEmpty() ? i : i - stack.peek() - 1; // span between boundaries
        // process height * width
    }
    stack.push(i);
}

// 3. MONOTONE DEQUE – sliding window max/min
// MENTAL MODEL: the front is the current window's answer; anything a newer-and-bigger value beats is dead weight.
// WHEN: "max/min of every window of size k"
// Deque stores INDICES; values at those indices are decreasing front-to-back (for max)
Deque<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!queue.isEmpty() && nums[queue.peekLast()] <= nums[i])
        queue.pollLast(); // remove smaller elements – they can never be window max
    queue.offerLast(i);
    if (queue.peekFirst() < i - k + 1) queue.pollFirst(); // evict out-of-window elements
    if (i >= k - 1) result[i - k + 1] = nums[queue.peekFirst()];
}

// 4. PRIORITY QUEUE
PriorityQueue<Integer> minPQ = new PriorityQueue<>(); // min-heap (default)
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
```

```

PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // custom: sort by index 1
pq.offer(x); // add
pq.poll(); // remove and return min/max
pq.peek(); // view min/max without removing

// 5. TWO HEAPS – maintain median in O(log n)
// MENTAL MODEL: split the data at the median; the median is always sitting on the two heaps' tops.
// WHEN: "running median", "median of a data stream"
// maxHeap = lower half (smaller numbers), minHeap = upper half (larger numbers)
// Invariant: maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size() + 1
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
void addNum(int num) {
    maxHeap.offer(num);
    minHeap.offer(maxHeap.poll()); // push one to minHeap (possibly num itself)
    if (maxHeap.size() < minHeap.size())
        maxHeap.offer(minHeap.poll()); // rebalance: maxHeap always ≥ minHeap in size
}
double findMedian() {
    return maxHeap.size() > minHeap.size()
        ? maxHeap.peek()
        : (maxHeap.peek() + minHeap.peek()) / 2.0;
}

```

Decreasing vs Increasing

DECREASING stack (pop when current > top): finds NEXT GREATER element for each popped index
 INCREASING stack (pop when current < top): finds NEXT SMALLER element – used for span width

Both store INDICES (not values) so you can compute distances and widths.

Structure Chooser

Goal	Structure	Why
Next greater/smaller element	Monotone stack	O(n) total — each element pushed/popped once
Sliding window max/min	Monotone deque	Front = window extreme; stale indices evicted
Global min/max, k-th element	PriorityQueue	O(log n) per op
Running median	Two heaps	Split at median; each heap half is O(log n)
O(1) stack minimum	Aux min-stack	Mirror min value at every level
Nested structure (brackets, k-repeats)	Two stacks	One for counts, one for strings

Deque API Cheat Sheet

One class, three roles. `ArrayDeque` adds/removes at *both* ends, so stack, queue, and deque are just usage patterns of the same structure — the only difference is **which end you remove from**. Always declare it `Deque<Integer> queue = new ArrayDeque<>();` (never the legacy `Stack` class, which is synchronized and `Vector`-backed; and `ArrayDeque` beats `LinkedList` as a queue). Name the variable for the *role* it plays (`stack` / `queue`).

```

addFirst / push          addLast / offer
    ↓                    ↓
[ FRONT ..... BACK ]
    ↑                    ↑
pollFirst / pop          pollLast

```

STACK (LIFO): add + remove at FRONT → newest out first
 QUEUE (FIFO): add at BACK, remove at FRONT → oldest out first

Intent	Stack (LIFO)	Queue (FIFO)
Add	push(x) → addFirst	offer(x) → addLast
Remove	pop() → removeFirst	poll() → removeFirst
Peek	peek() → peekFirst	peek() → peekFirst

Both **remove from the front** — the discipline comes from *where you add*: stack adds to the front (so newest is popped = LIFO), queue adds to the back (so oldest is polled = FIFO).

```
Deque<Integer> queue = new ArrayDeque<>();

// As STACK (LIFO):  push()/pop()/peek() → operate on FRONT
queue.push(x);      // = offerFirst
queue.pop();        // = pollFirst
queue.peek();       // = peekFirst

// As QUEUE (FIFO):  offer()/poll()/peek() → back-in, front-out
queue.offerLast(x);
queue.pollFirst();
queue.peekFirst();

// As DEQUE:         explicit first/last (e.g. monotone sliding-window)
queue.offerFirst(x); queue.pollFirst(); queue.peekFirst();
queue.offerLast(x);  queue.pollLast();  queue.peekLast();
```

BFS (Tree) bfs.md

Core structure: `Queue<TreeNode> queue = new ArrayDeque<>()` .
The only decision: do you need to know **which level** a node is on?

The Two Templates

```
// 1. LEVEL-AWARE PROCESSING – snapshot size to make per-level decisions
// MENTAL MODEL: the queue holds exactly one full level; freeze its size, then drain just that many.
// WHEN: "level by level", "per row", "rightmost/leftmost in level", "min depth"
Queue<TreeNode> queue = new ArrayDeque<>();
queue.offer(root);
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();      // ← snapshot: all nodes currently in queue = this level
    level++;
    for (int i = 0; i < size; i++) {
        TreeNode node = queue.poll();
        // process node (know: level, i==0 → first, i==size-1 → last)
        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}

// 2. WITHOUT LEVEL TRACKING – simple node-by-node
// MENTAL MODEL: just visit every node in BFS order; you don't care which level it's on.
Queue<TreeNode> queue = new ArrayDeque<>();
queue.offer(root);
while (!queue.isEmpty()) {
    TreeNode node = queue.poll();
    // process node
    if (node.left != null) queue.offer(node.left);
    if (node.right != null) queue.offer(node.right);
}
```

Almost all tree BFS problems use **Template 1** — you nearly always need `size` to know when a level ends.

When to Use — Signal → Pattern

Problem signal phrase	Pattern / group
"level order", "values per level", "each row"	Group 1 — collect entire level
"average / max / sum of each level"	Group 2 — aggregate per level
"right side view" (last in level), "bottom-left" (first in level)	Group 2 — record boundary node (i==size-1 / i==0)
"minimum depth", "first level that satisfies X"	Group 3 — early return on condition
"connect next pointers at the same level"	Group 4 — wire nodes within a level
"same depth, different parent" (cousins), per-node metadata	Group 5 — track per-node metadata

"maximum width of a level" (counting null gaps)	Group 6 — position indexing
no level info needed, just visit every node	Template 2 — node-by-node

The One Rule — Snapshot Level Size First

```
Always snapshot the level size BEFORE the inner for loop:
int size = queue.size();
for (int i = 0; i < size; i++) { ... }
```

Without the snapshot, newly added children mix with the current level
and `i == size-1 / i == 0` checks become meaningless.

Quick Reference Table

#	Name	Description	Level var	Key action per level	Return
102	Level Order	All nodes level by level	No	collect all <code>node.val</code>	<code>List<List<Integer>></code>
107	Level Order II	Same but bottom-up	No	<code>result.addFirst(level)</code>	<code>List<List<Integer>></code>
103	Zigzag Level Order	Alternate direction each level	<code>leftToRight</code> flag	<code>addLast / addFirst</code>	<code>List<List<Integer>></code>
637	Average of Levels	Mean value at each level	No	<code>sum / size</code>	<code>List<Double></code>
515	Largest Value Per Row	Max value at each level	No	<code>Math.max</code> over level	<code>List<Integer></code>
1161	Maximum Level Sum	Level number with greatest sum	<code>level</code> counter	<code>compare sum > maxSum</code>	<code>int</code> (level #)
199	Right Side View	Rightmost node at each level	No	record when <code>i == size-1</code>	<code>List<Integer></code>
513	Bottom Left Value	First node at the last level	No	record when <code>i == 0</code>	<code>int</code>
111	Minimum Depth	Depth of the first leaf reached	<code>depth</code> counter	return when leaf found	<code>int</code>
116	Next Right Pointers I	Connect nodes at same level (perfect tree)	No	<code>previous.next = node</code>	<code>Node</code>
117	Next Right Pointers II	Connect nodes at same level (any tree)	No	<code>previous.next = node</code>	<code>Node</code>
662	Maximum Width of Binary Tree	Max width of any level (between leftmost and rightmost non-null nodes, counting nulls)	BFS with position index per node; <code>width = lastPos - firstPos + 1</code> per level	Position indexing: assign position <i>2pos (left) and 2pos+1 (right)</i> ; normalize to avoid overflow	<code>O(n)</code>
993	Cousins in Binary Tree	Same depth, different parents?	No	track parent of <code>x</code> and <code>y</code>	<code>boolean</code>

Pattern Summary

What changes per level	Key variable	Example problems
Collect all values	<code>List<Integer> level</code>	102, 107, 103
Compute aggregate	<code>int sum / int max</code>	637, 515, 1161
Record boundary node	check <code>i == 0</code> or <code>i == size-1</code>	199, 513
Return on first match	<code>return depth</code>	111
Wire nodes together	<code>Node previous</code>	116, 117
Track parent metadata	<code>TreeNode xParent, yParent</code>	993

The One Rule (recap)

Snapshot the level size **before** the inner `for` loop — see "The One Rule" at the top of this file for the full explanation.

DFS / Backtracking

dfs.md

DFS appears in five forms. The pattern determines naming, return type, and where the answer is built.

When to Use — Signal → Pattern

Problem signal phrase	Pattern / template
"height / depth / sum OF the subtree", combine results upward	#1 Process children first, combine at node (post-order)
"root-to-leaf path", "number built along the path"	#2 Process node first, pass state down (pre-order)
"longest/best path that may bend through a node" (spans both arms)	#3 Tree global answer
"count islands / largest region / flood fill" on a grid	#4 Grid DFS
"can finish all", "detect cycle", "valid course order" (directed)	#5 Graph DFS 3-color
"all subsets / permutations / combinations", "find every way"	#6 Backtracking
"valid BST", "kth smallest", "closest value", "successor"	BST templates (pass bounds / inorder / binary search)

Quick Reference Table

#	Name	Description	Pattern	Returns	Global var
104	Max Depth	Maximum depth of binary tree	Tree — return up	depth int	No
543	Diameter	Longest path between any two nodes	Tree — return up	height int	Yes (diameter)
124	Max Path Sum	Max sum path between any two nodes	Tree — return up	best single-arm int	Yes (maxSum)
110	Balanced Tree	Is tree height-balanced?	Tree — return up	height or -1	No
226	Invert Tree	Mirror a binary tree	Tree — return up	TreeNode	No
112	Path Sum	Any root-to-leaf path summing to target?	Tree — pass down	boolean	No
113	Path Sum II	All root-to-leaf paths summing to target	Tree — pass down + backtrack	void	No
257	Binary Tree Paths	All root-to-leaf paths as strings	Tree — pass down + backtrack	void	No
129	Sum Root to Leaf	Sum of all root-to-leaf numbers	Tree — pass down	int	No
200	Number of Islands	Count connected land regions in grid	Grid DFS	void	No
695	Max Area of Island	Largest island area	Grid DFS	int (area)	No
207	Course Schedule	Can all courses be finished (no cycle)?	Graph DFS 3-color	boolean	No
210	Course Schedule II	Return valid course order	Graph DFS 3-color	void → post-order	No
46	Permutations	All permutations of distinct integers	Backtrack — no start, used[]	void	No
78	Subsets	All subsets of distinct integers	Backtrack — start, no used[]	void	No
39	Combination Sum	All combos summing to target (reuse allowed)	Backtrack — start, reuse i	void	No
40	Combination Sum II	All combos summing to target (no reuse)	Backtrack — start, skip dups	void	No
79	Word Search	Does word exist in board?	Grid backtrack — restore cell	boolean	No
131	Palindrome Partitioning	All palindrome partitions of string	Backtrack — start, check palindrome	void	No
98	Validate Binary Search Tree	Check if a binary tree is a valid BST	Pass min/max bounds down recursively; every node must satisfy both constraints	Pass bounds down: validate(node, min, max) with long bounds to handle INT_MIN/INT_MAX	O(n)

230	Kth Smallest Element in a BST	Find kth smallest element	Inorder traversal (left→node→right) counts ascending; stop at kth	Inorder + counter: global count/result; return early when count == k	O(k)
270	Closest BST Value	Find value in BST closest to target	BST search: go left if target < node, right if target > node; track closest	BST binary search: use BST structure to traverse in O(h)	O(h)
285	Inorder Successor in BST	Find the smallest node greater than p	BST traversal: if root > p it could be successor; else go right	BST successor search: save candidate when going left	O(h)
549	Binary Tree Longest Consecutive Sequence II	Longest consecutive path (increasing OR decreasing, may pass through node)	Post-order DFS returning {inc, dec} lengths per node	Two lengths returned: track both increasing and decreasing runs; combine at node	O(n)
687	Longest Univalue Path	Longest path where all nodes have same value	Post-order DFS; at each node, extend path from children only if values match	Conditional extend: leftPath = (left.val==node.val) ? left+1 : 0	O(n)
37	Sudoku Solver	Fill 9x9 grid so each row/col/3x3 box has 1-9	Backtracking; try 1-9 in each empty cell, validate, recurse	Constraint validation: check row, col, and 3x3 box before placing	O(9^m)
47	Permutations II	All unique permutations of array with duplicates	Backtracking + sort + used[]; skip used[i-1] duplicate	Skip duplicate at same level: if i>0 && nums[i]==nums[i-1] && !used[i-1] continue	O(n·n!)
51	N-Queens	All board placements where no queens attack	Backtracking by row; track cols, diag1 (r-c), diag2 (r+c) sets	Diagonal sets: r-c and r+c uniquely identify diagonals	O(n!)
60	Permutation Sequence	kth permutation of 1..n (1-indexed)	Math: factorial number system; pick each digit directly	Direct math, no backtracking: index = (k-1)/(n-1)!; remove used digit	O(n²)
90	Subsets II	All unique subsets of array with duplicates	Backtracking; sort first; skip i>start && nums[i]==nums[i-1]	Skip duplicate branches: sort + skip same value at same depth	O(n·2^n)
216	Combination Sum III	k numbers from 1-9 summing to n, no reuse	Backtracking with start index advancing i+1	Two constraints: track both count k and remaining sum	O(C(9,k))

All Templates

```
// 1. PROCESS CHILDREN FIRST, COMBINE AT NODE (post-order)
// MENTAL MODEL: each node trusts its children to return a finished answer, then merges them.
// WHEN: "what is the height/sum/property OF the subtree rooted here"
private int dfs(TreeNode node) {
    if (node == null) return BASE;           // 0 for depth, true for balanced
    int left = dfs(node.left);
    int right = dfs(node.right);
    // combine left, right, node.val → answer propagates up
    return ...;
}

// 2. PROCESS NODE FIRST, PASS STATE DOWN (pre-order)
// MENTAL MODEL: carry what you've seen so far down the path; the leaf has the full picture.
// WHEN: "root-to-leaf path", "running sum/number built along the way"
private void dfs(TreeNode node, int accumulated) {
    if (node == null) return;
    accumulated += node.val;                 // update state on way down
    if (isLeaf(node)) { /* record */ return; }
    dfs(node.left, accumulated);
    dfs(node.right, accumulated);
    // backtrack: no undo needed for primitive; undo List.add if using collection
}

// 3. TREE – GLOBAL ANSWER (return up the best single arm; update global across both arms)
// MENTAL MODEL: the answer may bend through a node using both arms, but only one arm can extend upward.
// WHEN: use when the best path spans across a node, not just up one arm.
int answer = Integer.MIN_VALUE;
private int dfs(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(0, dfs(node.left)); // clamp negative
    int right = Math.max(0, dfs(node.right));
    answer = Math.max(answer, left + right + node.val); // cross-node path
    return Math.max(left, right) + node.val; // single arm up
}

// 4. GRID DFS – mark visited by mutating grid; 4-directional flood fill
// MENTAL MODEL: stand on a cell, flood into all 4 neighbors, sinking each as you visit it.
// WHEN: "connected region / island / flood fill" on a grid
private void dfs(int[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (grid[r][c] != TARGET) return;
```

```

    grid[r][c] = VISITED;
    dfs(grid, r+1, c); dfs(grid, r-1, c);
    dfs(grid, r, c+1); dfs(grid, r, c-1);
}

// 5. GRAPH DFS - 3-color visited: 0=unseen 1=in-stack 2=done
// MENTAL MODEL: if you re-enter a node still on the current stack, you've looped back → cycle.
// WHEN: "can finish all", "detect cycle", "valid ordering" on a directed graph
int[] state;
private boolean dfs(int node) {
    if (state[node] == 1) return true;    // back edge → cycle
    if (state[node] == 2) return false;   // already safe
    state[node] = 1;
    for (int neighbor : graph.get(node))
        if (dfs(neighbor)) return true;
    state[node] = 2;
    return false;
}

// 6. BACKTRACKING - choose / recurse / undo
// MENTAL MODEL: build a candidate one element at a time; undo each choice to explore the next branch.
// WHEN: "all subsets / permutations / combinations", "find all ways"
private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (skipCondition(i)) continue;    // prune duplicates
        current.add(nums[i]);              // choose
        backtrack(i+1, current);           // next = i (reuse) or i+1 (no reuse)
        current.remove(current.size()-1);  // undo
    }
}

```

PROCESS CHILDREN FIRST, COMBINE AT NODE (post-order)

Recurse into children first, then combine results at the current node. Answer propagates upward.

PROCESS NODE FIRST, PASS STATE DOWN (pre-order)

State is built on the way **down** and consumed at leaves. Use backtracking when state is a mutable collection.

Grid DFS (4-directional flood fill)

Mark visited by overwriting the cell value. Return the accumulated property (count, area) or void.

Graph DFS — 3-Color Cycle Detection

Three states: **0** = unvisited, **1** = in current DFS stack (visiting), **2** = fully processed (safe).

A back edge (reaching a node with state **1**) means a cycle.

Template: choose → recurse → undo.

start controls which elements are eligible at each level. **current** is the in-progress candidate.

```

// MENTAL MODEL: grow one candidate by picking an element, recurse, then undo to try the next branch.
private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (skipCondition) continue;
        current.add(nums[i]);          // choose
        backtrack(i+1, current);       // nextStart = i (reuse) or i+1 (no reuse)
        current.remove(current.size() - 1); // undo
    }
}

```

Backtracking Decision Table

#	All orderings?	Reuse element?	Has duplicates?	Start index	Skip condition
46 Permutations	Yes — used[]	No	No	Loop 0..n, used[i]	used[i]
78 Subsets	No	No	No	start → i+1	None
39 Combination Sum	No	Yes	No	start → i	candidates[i] > remaining

40 Combination Sum II	No	No	Yes	start → i+1	i > start && nums[i] == nums[i-1]
131 Palindrome Part.	No	No	No	start → end	!isPalindrome(sub)

Tree DFS Summary

Pattern	When to use	Answer location
Return up	Combine children at node	return value bubbles up
Pass down	Accumulate state toward leaves	check at leaf
Global variable	Path spans across a node (not up)	int/long field, updated inside dfs

BST key property: `left subtree values < node.val < right subtree values` . This allows O(h) search/insert/delete by binary-searching the tree.

BST Template: Pass Bounds Down

```
// Validate BST by passing min/max constraints down the tree
boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false; // ← bounds check
    return validate(node.left, min, node.val)           // left: tighten max
        && validate(node.right, node.val, max);         // right: tighten min
}

// Call with: validate(root, Long.MIN_VALUE, Long.MAX_VALUE)
// Use Long to handle Integer.MIN_VALUE / Integer.MAX_VALUE edge cases
```

BST Template: Inorder Traversal = Sorted Order

```
// BST inorder (left → node → right) visits nodes in ascending order
void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);
    // process node (kth smallest: increment counter, return when k)
    inorder(node.right);
}
```

BST Template: Binary Search on Tree

```
// Navigate BST like binary search: go left or right based on comparison
TreeNode search(TreeNode root, int target) {
    while (root != null) {
        if (root.val == target) {
            return root;
        } else if (root.val < target) {
            root = root.right;
        } else {
            root = root.left;
        }
    }
    return null;
}
```

Graph `graph.md`

`Queue<Integer> queue = new ArrayDeque<>()` throughout.
Six algorithms — each with a canonical template and representative problems.

Complexity Legend

BFS / DFS	$O(V + E)$	visit each vertex and edge once
Topo sort (Kahn)	$O(V + E)$	each node enqueued once, each edge relaxed once
Union-Find	$\sim O(n \cdot \alpha(n)) \approx O(n)$	α = inverse Ackermann, effectively constant
Dijkstra	$O((V + E) \log V)$	non-negative weights ONLY
Bellman-Ford	$O(V \cdot E)$	allows negative weights; run k+1 rounds for a k-hop limit
Prim's MST	$O(E \log V)$	PQ of candidate edges

Quick Reference Table

#	Name	Algorithm	Graph type	Key structure	Return	Time	Space
994	Rotting Oranges	Multi-source BFS	Grid	Queue<int[]>	int (minutes)	$O(V+E)$	$O(V)$
542	01 Matrix	Multi-source BFS	Grid	Queue<int[]> + dist[][]	int[][]	$O(V+E)$	$O(V)$
127	Word Ladder	BFS on state	Implicit	Queue<String> + Set	int (steps)	$O(N \cdot L \cdot 26)$	$O(N \cdot L)$
130	Surrounded Regions	BFS flood fill	Grid	Queue<int[]>	void	$O(V+E)$	$O(V)$
417	Pacific Atlantic	Multi-source BFS x2	Grid	Queue<int[]> x2	List<List<int>>	$O(V+E)$	$O(V)$
207	Course Schedule	Topo sort (Kahn's)	Directed	int[] inDegree	boolean	$O(V+E)$	$O(V+E)$
210	Course Schedule II	Topo sort (Kahn's)	Directed	int[] inDegree	int[]	$O(V+E)$	$O(V+E)$
310	Minimum Height Trees	Topo sort (leaf trim)	Undirected	Set<Integer>[]	List<Integer>	$O(V+E)$	$O(V+E)$
547	Number of Provinces	Union Find	Undirected	int[] parent	int	$O(n^2 \cdot \alpha(n))$	$O(n)$
684	Redundant Connection	Union Find	Undirected	int[] parent	int[]	$O(n \cdot \alpha(n))$	$O(n)$
1319	Connect Network	Union Find	Undirected	int[] parent	int	$O(E \cdot \alpha(n))$	$O(n)$
743	Network Delay Time	Dijkstra	Directed weighted	PriorityQueue<int[]>	int	$O((V+E) \log V)$	$O(V+E)$
787	Cheapest Flights K Stops	Bellman-Ford (k rounds)	Directed weighted	int[] dist per round	int	$O(k \cdot E)$	$O(V)$
1514	Max Probability Path	Dijkstra (maximize)	Undirected weighted	PriorityQueue max-heap	double	$O((V+E) \log V)$	$O(V+E)$
785	Is Graph Bipartite?	BFS 2-coloring	Undirected	int[] color	boolean	$O(V+E)$	$O(V)$
1584	Min Cost Connect Points	Prim's MST	Complete graph	PriorityQueue<int[]>	int	$O(n^2 \log n)$	$O(n^2)$
261	Graph Valid Tree	Check if n nodes and edges form a valid tree	Union Find: valid tree has exactly n-1 edges AND no cycles	Tree check: edges.length == n-1 is necessary; UF detects cycles	boolean	$O(n \cdot \alpha(n))$	$O(n)$
269	Alien Dictionary	Determine character order from sorted alien words	Build directed graph from adjacent word pairs; topological sort	Topo sort on char graph: compare adjacent words letter-by-letter to extract ordering edges	String	$O(C)$ C=total chars	$O(1)$
286	Walls and Gates	Fill each empty room with distance to nearest gate	Multi-source BFS from all gates simultaneously	Multi-source BFS: seed queue with ALL gates at distance 0	void	$O(m \cdot n)$	$O(m \cdot n)$
323	Number of Connected Components	Count connected components in undirected graph	Union Find: union all edges; count distinct roots	Standard UF component count: start with n, decrement per successful union	int	$O(n \cdot \alpha(n))$	$O(n)$
721	Accounts Merge	Merge accounts sharing any common email	Union Find on emails; group by root	Union Find on strings: map each email to an id, union emails within an account	List<List<String>>	$O(n \cdot \alpha)$	$O(n)$

827	Making A Large Island	Flip one 0 to 1 to maximize island size	Label islands with id+size map; for each 0 sum unique neighbor island sizes +1	Two-pass labeling: first DFS-color islands, then test each 0	int	$O(m \cdot n)$	$O(m \cdot n)$
909	Snakes and Ladders	Min moves to reach last square (board with snakes/ladders)	BFS on flattened board; convert square number to (row,col) via boustrophedon	BFS on board cells: number→coordinate conversion for zigzag rows	int	$O(n^2)$	$O(n^2)$
1091	Shortest Path in Binary Matrix	Shortest 8-directional path top-left to bottom-right through 0s	BFS with 8 directions	8-directional BFS: include diagonals in dr/dc	int	$O(m \cdot n)$	$O(m \cdot n)$
1631	Path With Minimum Effort	Min effort path from top-left to bottom-right; effort = max abs difference along path	Dijkstra with effort[r][c] = min-so-far max diff; PQ sorted by effort	Modified Dijkstra: dist = max(currEffort, edgeCost) instead of sum	int	$O(m \cdot n \cdot \log(m \cdot n))$	$O(m \cdot n)$

Graph Representation

```
// Adjacency list (most common)
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]);    // omit for directed
}

// Weighted adjacency list: int[] = {neighbor, weight}
List<List<int[]>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges)
    graph.get(edge[0]).add(new int[]{edge[1], edge[2]});

// Grid 4-directional neighbors
int[] dr = {1, -1, 0, 0};
int[] dc = {0, 0, 1, -1};
```

All Six Templates

```
// 1. BFS – shortest path / level order
// MENTAL MODEL: explore in rings of equal distance, so the first time you reach a node is the shortest.
// WHEN: "fewest steps", "shortest path" on an unweighted graph
Queue<Integer> queue = new ArrayDeque<>();
boolean[] visited = new boolean[n];
queue.offer(start);
visited[start] = true;
while (!queue.isEmpty()) {
    int node = queue.poll();
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            queue.offer(neighbor);
        }
    }
}

// 2. TOPOLOGICAL SORT – Kahn's BFS
// MENTAL MODEL: peel off nodes with no remaining prerequisites; if any get stuck, a cycle exists.
// WHEN: "valid order", "dependencies/prerequisites", "detect cycle" on a directed graph
int[] inDegree = new int[n];
for (int[] edge : edges) inDegree[edge[1]]++;
Queue<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.offer(i);
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.poll();
    order.add(node);
    for (int neighbor : graph.get(node))
        if (--inDegree[neighbor] == 0) queue.offer(neighbor);
}
// order.size() == n → no cycle
```

```

// 3. UNION FIND
// MENTAL MODEL: each set points to one representative; merge sets by linking roots, query by finding roots.
// WHEN: "connected components", "is it already connected", "merge groups dynamically"
int[] parent, rank;
void init(int n) {
    parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression
    return parent[x];
}
boolean union(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

// 4. DIJKSTRA – shortest path (non-negative weights)
// MENTAL MODEL: greedily settle the closest unfinished node; non-negative weights guarantee it's final.
// WHEN: "shortest/cheapest path" with non-negative weights
int[] dist = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // min-heap on cost
pq.offer(new int[]{src, 0});
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], d = current[1];
    if (d > dist[node]) continue; // stale entry
    for (int[] nb : graph.get(node)) {
        int next = nb[0], w = nb[1];
        if (dist[node] + w < dist[next]) {
            dist[next] = dist[node] + w;
            pq.offer(new int[]{next, dist[next]});
        }
    }
}

// 5. BIPARTITE CHECK – BFS 2-coloring
// MENTAL MODEL: paint neighbors the opposite color; a neighbor that's already your color means an odd cycle.
// WHEN: "split into two groups", "2-colorable", "no edge within a group"
int[] color = new int[n];
Arrays.fill(color, -1);
Queue<Integer> queue = new ArrayDeque<>();
for (int start = 0; start < n; start++) {
    if (color[start] != -1) continue;
    color[start] = 0;
    queue.offer(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : graph.get(node)) {
            if (color[neighbor] == -1) { color[neighbor] = 1 - color[node]; queue.offer(neighbor); }
            else if (color[neighbor] == color[node]) return false; // conflict
        }
    }
}
return true;

// 6. PRIM'S MST – greedy, always add cheapest edge to growing tree
// MENTAL MODEL: grow one tree outward, each step absorbing the cheapest edge that reaches a new node.
// WHEN: "connect all nodes at minimum total cost"
boolean[] inMST = new boolean[n];
int[] minEdge = new int[n];
Arrays.fill(minEdge, Integer.MAX_VALUE);
minEdge[0] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
pq.offer(new int[]{0, 0});
int total = 0;
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], cost = current[1];
    if (inMST[node]) continue;
    inMST[node] = true;
    total += cost;
    for (int[] nb : graph.get(node))
        if (!inMST[nb[0]] && nb[1] < minEdge[nb[0]]) {
            minEdge[nb[0]] = nb[1];
            pq.offer(new int[]{nb[0], nb[1]});
        }
}

```

```
}  
}
```

Single-Source vs Multi-Source

Single-source: one starting node, find distances to all others
Multi-source: seed queue with ALL starting nodes at once → BFS fans out from all simultaneously
→ avoids nested loops, handles "nearest X" questions in $O(n)$

Core idea: repeatedly remove nodes with `inDegree == 0` (no prerequisites). If all nodes removed → no cycle. Post-order in DFS = reverse of `order` here.

Template (always the same three methods):

```
int[] parent, rank;  
  
void init(int n) {  
    parent = new int[n];  
    rank = new int[n];  
    for (int i = 0; i < n; i++) parent[i] = i;  
}  
  
int find(int x) {  
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression  
    return parent[x];  
}  
  
boolean union(int x, int y) { // returns false if already connected  
    int px = find(x), py = find(y);  
    if (px == py) return false;  
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }  
    parent[py] = px;  
    if (rank[px] == rank[py]) rank[px]++;  
    return true;  
}
```

Dijkstra vs Bellman-Ford

Weights:	Dijkstra Non-negative only	Bellman-Ford Any (handles negative)
Constraint:	—	k-hop limit (run k+1 rounds)
Time:	$O((V+E) \log V)$	$O(k \cdot E)$
When to use:	Standard shortest path	k-stop limit or negative weights
Key trick:	stale check: <code>d > dist[node]</code>	copy array each round (<code>temp[]</code>)

Algorithm Chooser

Signal in problem	Algorithm
Shortest path, unweighted / unit weight	BFS
Shortest path, non-negative weights	Dijkstra
Shortest path, k-hop limit	Bellman-Ford (k+1 rounds)
Shortest path, negative weights	Bellman-Ford (n-1 rounds)
Connected components, cycle detection	Union Find
Directed cycle / valid ordering	Topological Sort (Kahn's)
2-colorable graph	Bipartite BFS coloring
Connect all nodes minimum cost	Prim's / Kruskal's MST
"nearest X for all cells"	Multi-source BFS

Stale Entry Pattern (Dijkstra)

```
int[] current = pq.poll();
int node = current[0], d = current[1];
if (d > dist[node]) continue; // ← this node was already relaxed to a better distance
```

Without this check, stale entries in the heap cause re-processing and wrong results.
It replaces a `visited[]` array — simpler and correct because a better distance always wins.

Greedy `greedy.md`

Two interval templates plus non-interval greedy patterns.

Quick Reference Table

#	Name	Sort by	Greedy rule	Key condition	Return
435	Non-overlapping Intervals	end	keep earliest-ending non-overlapping	<code>start >= lastEnd</code> → keep	int (removed count)
452	Min Arrows to Burst Balloons	end	one arrow covers all overlapping	<code>start > arrowEnd</code> → new arrow	int (arrows)
646	Max Length Pair Chain	end	pick longest non-overlapping chain	<code>pair[0] > lastEnd</code> → pick	int (chain length)
56	Merge Intervals	start	extend or start fresh	<code>last.end < current.start</code> → new	int[][]
57	Insert Interval	(no sort)	3 phases: before / overlap / after	<code>current.end < new.start</code> or <code>current.start > new.end</code>	int[][]
253	Meeting Rooms II	start	reuse room if meeting already ended	<code>pq.peek() <= current.start</code> → reuse	int (rooms)
55	Jump Game	—	track max reachable index	<code>i > maxReach</code> → stuck	boolean
45	Jump Game II	—	BFS layers: extend window to farthest	<code>i == currentEnd</code> → jump	int (jumps)
763	Partition Labels	—	track last occurrence of each char	<code>i == end</code> → close partition	List
406	Queue Reconstruction by Height	height desc, k asc	insert each person at their k index	<code>result.add(p[1], p)</code>	int[][]
134	Gas Station	Can complete circular route starting at some gas station	Greedy: track total gas surplus; if tank goes negative reset start; valid if total ≥ 0	Circular reset: if tank < 0, start = i+1, reset tank	O(n)
252	Meeting Rooms	Can person attend all meetings (no overlap)?	Sort by start time; check if <code>intervals[i][0] < intervals[i-1][1]</code>	Simple overlap check: sort by start, compare adjacent	O(n log n)

Two Interval Templates

MERGE DIRECTION RULE (for merge-style problems like #56, where both keys work):
Sort by START → traverse FORWARD (i = 0 → n-1), extend the END of last kept
Sort by END → traverse BACKWARD (i = n-1 → 0), extend the START of last kept
These two are exact mirror images. See #56 for both written out side by side.
(NOTE: this mirror only applies to merging. Template 1 below sorts by end but still sweeps FORWARD — there the goal is counting, not merging.)

TEMPLATE 1 — Sort by END time, sweep forward, track end of last kept interval
Use when: maximizing count of non-overlapping intervals
Greedy insight: earliest-finishing interval leaves most room for future ones

TEMPLATE 2 — Sort by START time, sweep forward, merge when overlapping
Use when: merging/counting overlapping intervals
Greedy insight: processing in start order → each interval only needs to compare with the last merged result

```
// MENTAL MODEL: sort to expose a local greedy choice, then sweep once making the locally-best pick.
// WHEN: intervals + "max non-overlapping / min arrows / merge / min rooms", or jump/partition sweeps.
// TEMPLATE 1: Sort by end, compare start
Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // sort by END time
int lastEnd = Integer.MIN_VALUE;
for (int[] interval : intervals) {
    if (interval[0] >= lastEnd) { // start >= lastEnd -> no overlap -> keep
        lastEnd = interval[1];
        // count++ or add to result
    } else {
        // overlap -> skip (or count as removed)
    }
}

// TEMPLATE 2: Sort by start, compare end
Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by START time
List<int[]> result = new ArrayList<>();
for (int[] interval : intervals) {
    if (result.isEmpty() || result.get(result.size()-1)[1] < interval[0]) {
        result.add(interval); // no overlap -> new interval
    } else {
        result.get(result.size()-1)[1] = Math.max(result.get(result.size()-1)[1], interval[1]); // overlap -> extend end
    }
}

// TEMPLATE 3: Sort by start + min-heap of end times (multi-resource)
// USE WHEN counting simultaneous/overlapping resources - "minimum rooms", "max concurrent".
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap of active end times
for (int[] interval : intervals) {
    if (!pq.isEmpty() && pq.peek() <= interval[0])
        pq.poll(); // reuse: earliest-ending resource is free
    pq.offer(interval[1]); // occupy a resource until interval[1]
}
// pq.size() = resources needed
```

Sort-by-End vs Sort-by-Start+Heap

Goal:	Sort by End	Sort by Start + Heap
Tracks:	count non-overlapping lastEnd (scalar)	count simultaneous active all active end times (heap)
Question answered:	max intervals I can keep	min rooms (resources) needed
Examples:	#435, #452, #646	#253 Meeting Rooms

Greedy Pattern Identifier

Signal	Template
"minimum removal to make non-overlapping"	Sort by end, keep non-overlapping (Template 1)
"minimum resources (arrows, groups)"	Sort by end, count groups (Template 1)
"merge overlapping intervals"	Sort by start, extend or add (Template 2)
"minimum rooms/machines"	Sort by start + min-heap (Template 3)
"can you reach the end?"	Track max reachable index
"minimum steps to reach the end?"	BFS layers via currentEnd
"partition string by last occurrence"	Track last[] array + extend end
"reconstruct order by two constraints"	Sort by one constraint, insert by the other
"can you complete the circuit / circular route?"	Track cumulative tank; reset start when tank < 0
"can person attend all meetings (no overlap)?"	Sort by start, compare adjacent intervals

Dynamic Programming dynamic_programming.md

Six template families. Every problem maps to one loop skeleton and one dp-state definition.

Quick Reference Table

#	Name	Category	dp state	Init	Key transition
10	Regular Expression Matching	Match string s against pattern p with . and *	2D DP: $dp[i][j] = s[0..i]$ matches $p[0..j]$; '*' means zero or more of preceding char	Two cases: zero occurrence $dp[i][j-2]$, or match $dp[i-1][j]$ if chars match	$O(m \cdot n)$
70	Climbing Stairs	Linear 1D	$dp[i] = \text{ways to reach step } i$	$dp[1]=1, dp[2]=2$	$dp[i-1]+dp[i-2]$
198	House Robber	Linear 1D	$dp[i] = \text{max rob up to } i$	$dp[0]=\text{nums}[0]$	$\max(dp[i-1], dp[i-2]+\text{nums}[i])$
213	House Robber II	Linear 1D	same, two runs	skip first or skip last	max of two runs
300	LIS	Look-back 1D	$dp[i] = \text{LIS ending at } i$	$dp[i]=1$	$\max(dp[j]+1)$ if $\text{nums}[j] < \text{nums}[i]$
139	Word Break	Look-back 1D	$dp[i] = \text{can break } s[0..i)$	$dp[0]=\text{true}$	$dp[j] \ \&\& \ \text{word}[j..i)$ in dict
416	Partition Equal Subset	Knapsack 0/1	$dp[j] = \text{can make sum } j$	$dp[0]=\text{true}$	$dp[j] =dp[j-\text{num}]$
494	Target Sum	Knapsack 0/1	$dp[j] = \text{ways to reach sum } j$	$dp[0]=1$	$dp[j] += dp[j-\text{num}]$
474	Ones and Zeroes	Knapsack 0/1 2D	$dp[i][j] = \text{max strings with } i \text{ zeros } j \text{ ones}$	0	$\max(dp[i][j], dp[i-z][j-o]+1)$
322	Coin Change	Knapsack Unbounded	$dp[j] = \text{min coins for amount } j$	$dp[0]=0, \text{rest}=\text{INF}$	$\min(dp[j], dp[j-\text{coin}]+1)$
518	Coin Change II	Knapsack Unbounded	$dp[j] = \text{ways to make amount } j$	$dp[0]=1$	$dp[j] += dp[j-\text{coin}]$
583	Delete Operation for Two Strings	2D Sequence (LCS)	$dp[i][j] = \text{LCS of word1}[..i) \text{ and word2}[..j)$	$dp[0][*]=0$	match: +1, else $\max(\text{up}, \text{left})$
377	Combination Sum IV	Permutation count	$dp[j] = \text{ordered arrangements for } j$	$dp[0]=1$	target outer, nums inner
1143	LCS	2D Sequence	$dp[i][j] = \text{LCS of } s1[..i) \text{ and } s2[..j)$	$dp[0][*]=0$	match: +1, else $\max(\text{up}, \text{left})$
72	Edit Distance	2D Sequence	$dp[i][j] = \text{edits to convert } s1[..i) \text{ to } s2[..j)$	$dp[i][0]=i$	match: diag, else $1+\min 3$
87	Scramble String	Is s2 a scramble of s1 via recursive partition swaps?	3D DP/memo: $dp[i][j][\text{len}]$; try every split point, with and without swap	Split + optional swap: check both swapped and non-swapped partitions	$O(n^4)$
91	Decode Ways	Look-back 1D	$dp[i] = \text{ways to decode } s[0..i)$	$dp[0]=1, dp[1]=1$	$dp[i] += dp[i-1]$ if 1-digit valid; $dp[i] += dp[i-2]$ if 2-digit valid
96	Unique Binary Search Trees	1D DP (Catalan)	$dp[i] = \text{count of unique BSTs with } i \text{ nodes}$	$dp[0]=dp[1]=1$	$dp[i] += dp[j-1] * dp[i-j]$ for $j=1..i$
115	Distinct Subsequences	2D Sequence	$dp[i][j] = \text{ways } s[..i) \text{ contains } t[..j)$	$dp[i][0]=1$	skip+match
647	Palindromic Substrings	Interval	$dp[i][j] = \text{is } s[i..j) \text{ palindrome}$	$dp[i][i]=\text{true}$	ends-match && inner
516	Longest Palindromic Subseq	Interval	$dp[i][j] = \text{LPS length in } s[i..j)$	$dp[i][i]=1$	ends-match: +2, else max
312	Burst Balloons	Interval	$dp[i][j] = \text{max coins in open interval } (i,j)$	0	max over k: $dp[i][k] + \text{coin} + dp[k][j]$
62	Unique Paths	Grid	$dp[i][j] = \text{paths to reach } (i,j)$	first row/col=1	$dp[i-1][j] + dp[i][j-1]$
63	Unique Paths II	Grid	$dp[i][j] = \text{paths to reach } (i,j) \text{ with obstacles}$	first row/col=1 until obstacle	$dp[i-1][j] + dp[i][j-1]$ if no obstacle, else 0
64	Min Path Sum	Grid	$dp[i][j] = \text{min cost to reach } (i,j)$	first row/col cumulative	$\min(\text{up}, \text{left}) + \text{grid}[i][j]$
221	Maximal Square	Grid	$dp[i][j] = \text{side of largest all-1 square at } (i,j)$	0	$\min(\text{up}, \text{left}, \text{diag}) + 1$
279	Perfect Squares	Knapsack Unbounded	$dp[i] = \text{min squares to sum to } i$	$dp[0]=0, \text{rest}=\text{INF}$	$\min(dp[i], dp[i-jj]+1)$ for $jj \leq i$

329	Longest Increasing Path	Grid + Memo	memo[i][j] = LIP starting at (i,j)	0	max(dfs(neighbor))+1
368	Largest Divisible Subset	Look-back 1D (LIS-style)	dp[i] = max subset ending at nums[i]	dp[i]=1	max(dp[j]+1) if nums[i]%nums[j]==0
121	Best Time I	State Machine	hold/cash, at most 1 buy	hold=-p[0]	hold=max(hold,-p[i])
122	Best Time II	State Machine	hold/cash, unlimited	hold=-p[0]	hold=max(hold,cash-p[i])
123	Best Time III	State Machine	buy1/sell1/buy2/sell2	buy1=buy2=-p[0]	chained 4 states
309	Stock with Cooldown	State Machine	hold/cash/cooldown	hold=-p[0]	cooldown=prevCash
714	Stock with Fee	State Machine	hold/cash	hold=-p[0]	sell: hold+p[i]-fee
746	Min Cost Climbing Stairs	Linear 1D	dp[i] = min cost to reach step i	dp[0]=dp[1]=0	min(dp[i-1]+cost[i-1], dp[i-2]+cost[i-2])
931	Minimum Falling Path Sum	Grid DP	dp[i][j] = min falling path sum to (i,j)	dp[0]=matrix[0]	matrix[i][j] + min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])
1049	Last Stone Weight II	Min possible weight of last stone after smashing	Partition into two near-equal subsets; 0/1 knapsack on sum/2	Reduce to subset sum: minimize S1 - S2 = total - 2*maxSubsetSum/2	O(n*sum)
1312	Minimum Insertion Steps to Make Palindrome	Min insertions to make string palindrome	Interval DP: dp[i][j] = min insertions for s[i..j]; reduce to n - LPS	LPS reduction: answer = n - longestPalindromicSubsequence	O(n ²)

All Six Templates

```
// 1. LINEAR 1D – each cell depends on a fixed number of previous cells
// MENTAL MODEL: the answer at i is a fixed recipe over the last one or two answers.
// WHEN: "ways/cost to reach step i", "can't pick adjacent"
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++)
    dp[i] = f(dp[i-1], dp[i-2], ...);

// 2A. LOOK-BACK 1D – dp[i] depends on all j < i
// MENTAL MODEL: to finish position i, scan every earlier position j and extend the best valid one.
// WHEN: "longest increasing/chain ending here", "can the prefix be segmented"
int[] dp = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        dp[i] = f(dp[i], dp[j]); // e.g., LIS, word break
    }
}

// -----
// MENTAL MODEL: both are the same 2D recurrence collapsed to 1D.
// dp[i][j] = ways to make sum j using the first i items.
// After dropping the i dimension, dp[j-nums[i]] is read from:
// • the PREVIOUS row (item i not yet used) → 0/1, no reuse
// • the CURRENT row (item i already used) → unbounded, reuse
// Loop direction decides which one you read.
// Mnemonic: DESCENDING = Distinct (each once), ASCENDING = Again (reusable).
// Seed/operator picks the flavor:
// count → dp[0]=1, dp[j] += dp[j-w]
// max-value → dp[0]=0, dp[j] = Math.max(dp[j], dp[j-w] + v)
// -----

// 2B. KNAPSACK 0/1 – each item at most once
int[] dp = new int[target + 1];
dp[0] = 1; // 1 way to make 0: take nothing
for (int i = 0; i < nums.length; i++)
    for (int j = target; j >= nums[i]; j--) // ↓ DESCENDING
        dp[j] += dp[j - nums[i]];
// ~~~~~ j-nums[i] < j, not yet touched THIS pass
// ~~~~~ → still "previous row" → item i unused → no reuse

// 2C. KNAPSACK UNBOUNDED – each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++)
    for (int j = nums[i]; j <= target; j++) // ↑ ASCENDING
        dp[j] += dp[j - nums[i]];
// ~~~~~ j-nums[i] < j, ALREADY updated THIS pass
```

```
//
// → "current row" → item i can reappear → reuse

// 3. 2D SEQUENCE – two strings/arrays; i indexes one, j indexes the other
// MENTAL MODEL: compare the last char of each prefix – match consumes both, mismatch drops one side.
// WHEN: "compare two strings/arrays" (LCS, edit distance, subsequence count)
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (match(i, j)) {
            dp[i][j] = dp[i-1][j-1] + val;
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1]);
        }
    }
}

// 4. INTERVAL DP – i goes RIGHT to LEFT; j goes i+1 to end
// "shorter comes first": when computing dp[i][j], dp[i+1][*] is already done
// MENTAL MODEL: build answers for short ranges first, then combine them into longer ranges.
// WHEN: "palindrome substring/subseq", "merge/burst in a range", "split-point problems"
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base; // length-1 intervals
for (int i = n - 1; i >= 0; i--) { // ← right-to-left
    for (int j = i + 1; j < n; j++) {
        dp[i][j] = f(dp[i+1][j-1], dp[i+1][j], dp[i][j-1]);
    }
}

// 5. GRID DP – 4-directional neighbors
// MENTAL MODEL: each cell's answer accumulates from the cells you could have arrived from.
// WHEN: "paths / min-cost in a grid moving right+down", "largest square"
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1}; // DOWN UP RIGHT LEFT
// Standard grid (dag, only right/down):
int[][] dp = new int[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dp[i][j] = grid[i][j] + combine(dp[i-1][j], dp[i][j-1]);

// 6. STATE MACHINE – named states updated simultaneously each step
// MENTAL MODEL: track the best value in each named situation; each day every state updates from yesterday's states.
// WHEN: "buy/sell/hold", "limited transactions", "cooldown/fee"
int cash = 0, hold = -prices[0];
for (int i = 1; i < prices.length; i++) {
    cash = Math.max(cash, hold + prices[i]);
    hold = Math.max(hold, ??? - prices[i]); // ??? depends on problem
}
}
```

Knapsack Decision Tree

```
Want count or min/max?
├─ Count (dp[j] += dp[j - num])
│   ├── 0/1 (each item once):    j descending → #416, #494
│   ├── Unbounded (reuse):      j ascending → #518
│   └─ Ordered (permut):       target outer, nums inner → #377
├─ Min/Max (dp[j] = min/max(...))
│   ├── 0/1 minimize:          j descending → #474 (maximize)
│   └─ Unbounded minimize:     j ascending → #322
```

Canonical Templates

```
// — 0/1 KNAPSACK — each item at most once
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = target; j >= nums[i]; j--) // DESCENDING: sees OLD dp (before item i)
        dp[j] += dp[j - nums[i]];
}

// — UNBOUNDED KNAPSACK — each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = nums[i]; j <= target; j++) // ASCENDING: sees NEW dp (after item i already used)
        dp[j] += dp[j - nums[i]];
}
}
```

```
// — PERMUTATION COUNT — order matters (Combination Sum IV)
int[] dp = new int[target + 1];
dp[0] = 1;
for (int j = 1; j <= target; j++) {           // OUTER: target
    for (int num : nums) {                   // INNER: try all nums (not fixing order)
        if (j >= num) dp[j] += dp[j - num];
    }
}
```

Why descending for 0/1:

`dp[j - nums[i]]` is read **before** `dp[j]` is updated ($j > j - \text{nums}[i]$). So it always sees the dp state from before item `i` was considered — no item is used twice.

Why ascending for unbounded:

`dp[j - nums[i]]` was already updated in this same pass ($j - \text{nums}[i] < j$, processed earlier). This allows item `i` to be picked multiple times.

2D anchor: both loops are `dp[i][j] = dp[i-1][j] + dp[i-?][j-w]` collapsed to 1D — descending keeps `dp[j-w]` on the *previous* row (item `i` unused), ascending pulls it from the *current* row (item `i` reused).

Mnemonic: DESCENDING = **D**istinct (each item once), ASCENDING = **A**gain (reusable).

Flavor = seed + operator (same skeleton):

- count → `dp[0]=1` , `dp[j] += dp[j-w]`
- feasibility → `dp[0]=true` , `dp[j] = dp[j] || dp[j-w]`
- max-value → `dp[0]=0` , `dp[j] = Math.max(dp[j], dp[j-w] + v)`

Canonical Template

```
// i indexes string/array 1 (1-indexed), j indexes string/array 2
int[][] dp = new int[m + 1][n + 1];
// Init dp[0][*] and dp[*][0] based on problem
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i-1) == s2.charAt(j-1)) {
            dp[i][j] = dp[i-1][j-1] + 1;           // characters match
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
        }
    }
}
```

Canonical Template

Rule: `i` goes right-to-left (ensures shorter/inner intervals are computed first). `j` goes left-to-right from `i+1`. When computing `dp[i][j]`, all `dp[i'][j']` with $i' > i$ or $j' < j$ are already done.

```
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base_case; // single elements
for (int i = n - 1; i >= 0; i--) {               // right-to-left
    for (int j = i + 1; j < n; j++) {             // left-to-right from i+1
        // dp[i][j] can safely use dp[i+1][j], dp[i][j-1], dp[i+1][j-1]
        dp[i][j] = ...;
    }
}
```

Alternative (length-based): Equivalent, sometimes clearer for k-partition problems:

```
for (int len = 2; len <= n; len++) {             // outer: length of interval
    for (int i = 0; i + len - 1 < n; i++) {       // inner: left endpoint
        int j = i + len - 1;
        dp[i][j] = ...;
    }
}
```

```
    }  
}
```

Canonical Template

```
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};    // DOWN UP RIGHT LEFT  
  
// Standard grid (only move right/down - DAG, no cycle):  
int[][] dp = new int[rows][cols];  
// initialize first row and first column  
for (int i = 0; i < rows; i++)  
    for (int j = 0; j < cols; j++)  
        dp[i][j] = grid[i][j] + f(dp[i-1][j], dp[i][j-1]);  
  
// All-direction grid (memoized DFS for increasing/decreasing paths):  
int[][] memo = new int[rows][cols];  
for (int i = 0; i < rows; i++)  
    for (int j = 0; j < cols; j++)  
        dfs(grid, i, j, memo, dr, dc);
```

Canonical States

cash = max profit when NOT holding (free to buy or idle)
hold = max profit when HOLDING stock (bought it at some cost)
cooldown = max profit right after selling (can't buy today)

All Five Transitions

	buy	sell	re-buy condition
#121 (1 tx):	hold = max(hold, -price)		no re-buy (ignore cash)
#122 (unlimited):	hold = max(hold, cash - price)		re-buy with full cash
#123 (2 tx):	chain: buy2 uses sell1 cash		4 states chained
#309 (cooldown):	hold = max(hold, cooldown - price)		must wait 1 day
#714 (fee):	hold = max(hold, cash - price)		pay fee on sell

Stock Problems — Differences at a Glance

Problem	hold update	cash update	Extra state
#121	max(hold, -price)	max(cash, hold+price)	none
#122	max(hold, cash-price)	max(cash, hold+price)	none
#123	4 variables, chained	-	sell1 feeds buy2
#309	max(hold, cooldown-price)	max(cash, hold+price)	cooldown = prevCash
#714	max(hold, cash-price)	max(cash, hold+price-fee)	none

Template Chooser

Signal in problem	Template
dp[i] depends on dp[i-1], dp[i-2]	Linear 1D — fixed lookback
dp[i] depends on all dp[j] where j < i	Look-back 1D — O(n²) inner loop
Each item used at most once, sum target	Knapsack 0/1 — j descending
Items reusable, sum target, order irrelevant	Knapsack Unbounded — j ascending
Items reusable, order matters (permutations)	Permutation count — target outer
Two strings/arrays, compare characters	2D Sequence DP
Palindrome, substring merge, balloon burst	Interval DP — i right-to-left
Grid, only right/down movement	Grid DP — cumulative from edges
Grid, all 4 directions, monotone constraint	DFS + memo (no cycle in DAG)

Trie

trie.md

Quick Reference Table

#	Name	Description	Algorithm	Variation from Template	Time	Space
208	Implement Trie	insert / search / startsWith	Standard insert + traverse	Standard	O(k) per op	O(n·k)
211	Add and Search Words	Like Trie but . matches any letter	Insert standard; search with DFS for .	DFS for wildcard: . triggers recursive search over all children	O(k) insert, O(26^k) search worst	O(n·k)
212	Word Search II	Find all words from list that exist in grid	Trie built from words + DFS grid backtracking	Grid DFS with Trie pruning: cut branch when no Trie path matches	O(M·N·4·3^(L-1))	O(total chars)
648	Replace Words	Replace each word with its shortest root in dictionary	Trie + early return on first isEnd hit	Early-exit: return prefix immediately when isEnd found	O(n·k) build + O(sentence) search	O(dict·k)
745	Prefix and Suffix Search	Given words list, find index of word with given prefix AND suffix	Build Trie keyed by "prefix#suffix" for all prefix-suffix pairs of each word; O(1) query	Precompute all pairs: for each word, store every (prefix## suffix) combo in HashMap	O(W·L²) build, O(p+s) query	O(W·L²)
1268	Search Suggestions System	For each prefix, return 3 lexicographic suggestions	Trie; store up to 3 words at each node during insert	Store words at node: each node caches its top-3 suggestions	O(n·k log n) build + O(k) search	O(n·k)

When to Use Trie (vs HashMap/Array)

Signal	Use
Prefix queries / startsWith / autocomplete	Trie — shares prefixes, prefix lookup is O(k)
Wildcard match (. matches any char)	Trie — DFS branches over children
Prune a search space by shared prefixes (e.g. Word Search II)	Trie — dead branches cut early
Exact-key lookup only (no prefix logic)	HashMap — O(k) hash is simpler and faster, no node overhead

Canonical Template

```
// MENTAL MODEL: every node is a shared prefix; walking down spells a word, so common prefixes are stored once.
// WHEN: "prefix / startsWith / autocomplete", "wildcard match", "prune a search by shared prefixes"

// — TRIE NODE —
// Array vs Map TrieNode mental model:
//   array = O(1) per char, fixed 26 letters, wastes space when sparse
//   HashMap = variable alphabet, smaller for sparse, hashing overhead
class TrieNode {
    TrieNode[] children = new TrieNode[26];    // array-based: O(1) access, O(26) per node
    boolean isEnd = false;
}
// Alternative: Map-based (for non-lowercase or variable alphabets)
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isEnd = false;
}

// — INSERT —
void insert(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) node.children[idx] = new TrieNode();
        node = node.children[idx];
    }
    node.isEnd = true;
}

// — SEARCH (exact match) —
```

```

boolean search(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) return false;
        node = node.children[idx];
    }
    return node.isEnd;    // must end at a marked word boundary
}

// — STARTS WITH (prefix match) —
boolean startsWith(TrieNode root, String prefix) {
    TrieNode node = root;
    for (char c : prefix.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) return false;
        node = node.children[idx];
    }
    return true;          // ← difference from search: don't check node.isEnd
}

```

Variations

```

// VARIATION 1: DFS for wildcard '.' (Add and Search Words)
// When char is '.', try all 26 children recursively instead of following one path.
boolean dfs(TrieNode node, String word, int i) {
    if (i == word.length()) return node.isEnd;
    char c = word.charAt(i);
    if (c == '.') {
        for (TrieNode child : node.children) // ← VARIATION: try all children
            if (child != null && dfs(child, word, i+1)) return true;
        return false;
    }
    int idx = c - 'a';
    if (node.children[idx] == null) return false;
    return dfs(node.children[idx], word, i + 1);
}

// VARIATION 2: early-exit on isEnd (Replace Words – find shortest root)
String findRoot(TrieNode root, String word) {
    TrieNode node = root;
    for (int i = 0; i < word.length(); i++) {
        int idx = word.charAt(i) - 'a';
        if (node.children[idx] == null) return word;    // no root found → return original
        node = node.children[idx];
        if (node.isEnd) return word.substring(0, i+1); // ← VARIATION: return at first root hit
    }
    return word;
}

// VARIATION 3: store words at each node (Search Suggestions – top-3 per prefix)
// During insert, each node on the path caches the word (up to 3).
// Products must be inserted in sorted order → first 3 are lexicographically smallest.
void insert(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) node.children[idx] = new TrieNode();
        node = node.children[idx];
        if (node.words.size() < 3) node.words.add(word); // ← VARIATION: cache at every node
    }
    node.isEnd = true;
}

// VARIATION 4: Trie + grid DFS pruning (Word Search II)
// Build Trie from word list. DFS the grid; at each cell, check if current path
// follows a Trie path. Prune entire branch if no Trie node exists.
void dfs(char[][] board, int i, int j, TrieNode node, StringBuilder path, Set<String> result) {
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) return;
    char c = board[i][j];
    if (c == '#' || node.children[c - 'a'] == null) return; // ← VARIATION: Trie prune
    node = node.children[c - 'a'];
    path.append(c);
    if (node.isEnd) result.add(path.toString());
    board[i][j] = '#';
    dfs(board, i+1, j, node, path, result);
    dfs(board, i-1, j, node, path, result);
    dfs(board, i, j+1, node, path, result);
    dfs(board, i, j-1, node, path, result);
}

```

```
board[i][j] = c;
path.deleteCharAt(path.length() - 1);
}
```

search vs startsWith — One Line Difference

```
search:      return node.isEnd;    // must land on a word boundary
startsWith:  return true;          // any node reached = valid prefix
```

Trie vs HashMap vs Sorting — When to Use Trie

Operation	HashMap	Sorting	Trie
Exact key lookup	O(k) ✓	O(log n · k)	O(k)
Prefix check	O(k) per key	O(log n)	O(k) ✓
All words with prefix	O(n · k)	O(log n + result)	O(k + result) ✓
Wildcard match	Not supported	Not supported	O(26^k) with DFS ✓
Space	O(n · k)	O(n · k)	O(n · k) alphabet-dependent

Use Trie when: prefix queries, wildcard search, or pruning a search space by shared prefixes (Word Search II).

Insert → Search — Structural Parallel

Both traverse the same path. Insert creates nodes; search reads them.
The only decision point after traversal:

```
search:      return node.isEnd      ← complete word?
startsWith:  return true            ← any prefix is fine
findRoot:    return early if isEnd  ← want the shallowest word boundary
cache words: node.words.add(word)   ← accumulate at every node during insert
```

Bit Manipulation bit_manipulation.md

Quick Reference Table

#	Name	Description	Algorithm	Variation from Template	Time	Space
136	Single Number	One element appears once; rest appear twice	XOR all — pairs cancel	Standard	O(n)	O(1)
137	Single Number II	One element appears once; rest appear three times	XOR state machine with ones, twos	mod 3 : two-state machine instead of simple XOR	O(n)	O(1)
260	Single Number III	Two unique elements; rest appear twice	XOR → split by lowest-diff bit	Split : use xor & (~xor) to separate into two groups	O(n)	O(1)
191	Number of 1 Bits	Count set bits in n	Brian Kernighan: n &= (n-1) loop	Standard bit clear	O(k) k=set bits	O(1)
338	Counting Bits	Return number of 1 bits for every i in 0..n	DP: dp[i] = dp[i >> 1] + (i & 1)	DP relation : right-shift halves the number	O(n)	O(n)
268	Missing Number	Missing number in [0, n]	XOR indices with values	Index XOR : XOR i with nums[i] instead of just nums	O(n)	O(1)
190	Reverse Bits	Reverse all 32 bits of an integer	Shift-and-OR loop 32 times	Fixed 32 iterations instead of until zero	O(32)	O(1)
201	Bitwise AND of Numbers Range	AND of all numbers in [left, right]	Right-shift both until equal; shift back	Common prefix : find shared high bits	O(log n)	O(1)
371	Sum of Two Integers	Add without + or -	XOR = sum without carry; AND << 1 = carry	Carry loop : separate sum and carry bits	O(32)	O(1)

318	Max Product of Word Lengths	Max product of word lengths with no shared letters	Precompute 26-bit mask per word; check <code>mask[i] & mask[j] == 0</code>	Bitmask as set: 26 bits represent letter presence	$O(n^2)$	$O(n)$
231	Power of Two	Check if n is a power of two	<code>n > 0 && (n & (n-1)) == 0</code>	Single bit check: power of 2 has exactly one set bit	$O(1)$	$O(1)$
287	Find the Duplicate Number	Find duplicate in <code>nums[1..n]</code> with <code>n+1</code> elements; no extra space	Floyd's cycle detection: treat array as linked list, <code>nums[i] → next node</code>	Floyd's cycle: not XOR; use slow/fast pointer cycle detection	$O(n)$	$O(1)$
342	Power of Four	Check if n is a power of four	Power of 2 AND the single set bit is at an even position (0,2,4,...)	Even bit position: check <code>(n & 0xAAAAAAAA) == 0</code> after power-of-2 check	$O(1)$	$O(1)$

Canonical Template

```
// MENTAL MODEL: bits are a set/counter – XOR cancels duplicates, AND/OR/shift edit individual bits.
// WHEN: "appears once/odd among pairs", "count set bits", "no shared letters", "power of 2/4".
// — XOR CANCEL — pairs cancel; lone element survives
int result = 0;
for (int num : nums) result ^= num;

// — BIT OPERATIONS —
boolean isSet = ((n >> i) & 1) == 1; // check if bit i is set
int set = n | (1 << i); // set bit i
int clear = n & ~(1 << i); // clear bit i
int toggle = n ^ (1 << i); // toggle bit i
int lowest = n & (-n); // isolate lowest set bit
boolean isPow2 = n > 0 && (n & (n-1)) == 0;

// — COUNT SET BITS — Brian Kernighan: each iteration removes one set bit
// WHY n & (n-1) clears the lowest set bit:
// n = ...1000 (lowest set bit at this position)
// n-1 = ...0111 (borrow flips that bit to 0 and all lower bits to 1)
// n&(n-1)=...0000 (AND keeps higher bits, wipes the lowest set bit + below)
int count = 0;
while (n != 0) { count++; n = (n - 1); } // n & (n-1) clears lowest set bit

// — BITMASK AS SET — 26-bit integer represents presence of 26 letters
int mask = 0;
for (char c : word.toCharArray()) mask |= (1 << (c - 'a'));
// check no overlap between two words:
if ((mask[i] & mask[j]) == 0) { /* no shared letter */ }
```

Variations

```
// VARIATION 1: mod-3 state machine (Single Number II)
// MENTAL MODEL: extend XOR from mod-2 (cancel pairs) to mod-3 (cancel triples);
// ones/twos track bits seen 1 or 2 times mod 3.
// Instead of XOR canceling pairs (mod 2), cancel triples (mod 3)
// ones = bits seen exactly 1 mod 3 times; twos = bits seen exactly 2 mod 3 times
int ones = 0, twos = 0;
for (int num : nums) {
    ones = (ones ^ num) & ~twos; // ← add to ones if not already in twos
    twos = (twos ^ num) & ~ones; // ← add to twos if not already in ones
}
// After all: ones holds the element appearing once (0 mod 3 remainder = survive)

// VARIATION 2: split XOR into two groups (Single Number III)
// Two unique elements a, b. XOR of all = a ^ b.
// Use lowest set bit of (a^b) to split nums into two groups → each group has one unique.
int xor = 0;
for (int num : nums) xor ^= num; // xor = a ^ b
int diff = xor & (-xor); // ← lowest set bit that differs between a and b
int a = 0;
for (int num : nums)
    if ((num & diff) != 0) a ^= num; // ← XOR one group → one unique element
int b = xor ^ a; // ← the other unique element

// VARIATION 3: DP relation (Counting Bits)
// dp[i] = dp[i/2] + last bit of i
// i >> 1 = i without last bit (same or fewer ones); last bit = i & 1
int[] dp = new int[n + 1];
for (int i = 1; i <= n; i++)
```



```

    dp[i] = dp[i >> 1] + (i & 1);    // ← right-shift halves the problem

// VARIATION 4: common prefix (Bitwise AND of Range)
// AND of any range [left, right]: bits that differ between left and right get cleared.
// Right-shift both until equal → find shared high-bit prefix → shift back.
int shift = 0;
while (left != right) { left >>= 1; right >>= 1; shift++; }
return left << shift;                // ← shift back to original magnitude

// VARIATION 5: carry loop (Sum of Two Integers)
// XOR gives bit sum without carry; AND shifted left gives carry.
// Repeat until no carry remains.
int add(int a, int b) {
    while (b != 0) {
        int carry = a & b;           // ← carry bits
        a = a ^ b;                   // ← partial sum (no carry)
        b = carry << 1;              // ← carry propagated one position left
    }
    return a;
}

```

Bit Trick Cheat Sheet

```

n & (n-1)           // clear lowest set bit → count bits, check power of 2
n & (-n)            // isolate lowest set bit → split into groups
n ^ n = 0           // same value cancels → XOR pairs
n ^ 0 = n           // identity → XOR with index
(n >> i) & 1         // check bit i
n | (1 << i)         // set bit i
n & ~(1 << i)        // clear bit i
n ^ (1 << i)         // toggle bit i
i >> 1              // divide by 2 (fast)
i & 1              // last bit (0 = even, 1 = odd)

// --- Divisibility by powers of two ---
(n & 1) == 0        // divisible by 2 (even) ← LSB is the 2^0 place
(n & 1) == 1        // NOT divisible by 2 (odd)
(n & ((1 << k) - 1)) // remainder of n / 2^k ; == 0 means divisible by 2^k
(n & 3) == 0        // divisible by 4 (k = 2)
(n & 7) == 0        // divisible by 8 (k = 3)
// Note: n & 1 is safe for NEGATIVE numbers (two's complement) where
//      n % 2 returns -1 for odd negatives. Prefer & 1 for parity.

```

Pattern Chooser

Signal	Technique	Time
"appears odd/once, rest appear even/twice"	XOR all	O(n)
"appears once, rest appear 3 times"	mod-3 state machine (ones, twos)	O(n)
"two unique elements"	XOR → split by xor & (−xor)	O(n)
Count set bits	Brian Kernighan n &= (n-1)	O(k) k = set bits
Count bits for all 0..n	DP: dp[i] = dp[i>>1] + (i&1)	O(n)
AND of a range	Right-shift to find common prefix	O(log n)
Add without +	XOR + carry loop	O(1) (32 iters)
"no shared characters between two strings"	26-bit bitmask, check AND == 0	O(n²) pairwise
"is n a power of two?"	n > 0 && (n & (n-1)) == 0	O(1)
"is n a power of four?"	Power of 2 AND set bit at even position: (n & 0xAAAAAAAA) == 0	O(1)
"find duplicate, no extra space, no modification"	Floyd's cycle detection on array-as-linked-list	O(n)

Quick Reference Table

#	Name	Description	Algorithm	Variation from Template	Time	Space
146	LRU Cache	O(1) get/put with LRU eviction	HashMap + doubly linked list; move accessed node to front	Standard	O(1) get/put	O(capacity)
460	LFU Cache	O(1) get/put with LFU eviction	HashMap<key→val>, HashMap<key→freq>, HashMap<freq→LinkedHashSet>; track minFreq	LFU: extra frequency map + minFreq variable	O(1) all ops	O(capacity)
380	Insert Delete GetRandom O(1)	Insert, remove, getRandom all in O(1)	HashMap<val→index> + ArrayList; swap-with-last on delete	Swap-to-end: on remove, swap target with last element to maintain dense array	O(1) all ops	O(n)

Canonical Template — HashMap + Doubly Linked List (LRU)

```
// MENTAL MODEL: HashMap gives O(1) lookup; the doubly linked list orders by recency so the LRU is always one hop from tail.
// WHEN: "O(1) get/put with eviction by recency or frequency"

// CacheNode for doubly linked list (named CacheNode to distinguish from ListNode contexts)
class CacheNode {
    int key, value;
    CacheNode previous, next;
    CacheNode(int key, int value) { this.key = key; this.value = value; }
}

// — SENTINEL NODES — head (MRU side) ↔ ... ↔ tail (LRU side)
CacheNode head = new CacheNode(0, 0), tail = new CacheNode(0, 0);
// Initialize: head <=> tail
head.next = tail; tail.previous = head;

// — REMOVE NODE —
void remove(CacheNode node) {
    node.previous.next = node.next;
    node.next.previous = node.previous;
}

// — INSERT AFTER HEAD (= mark as MRU) —
void insertAfterHead(CacheNode node) {
    node.next = head.next;
    node.previous = head;
    head.next.previous = node;
    head.next = node;
}

// — LRU EVICTION — tail.previous is LRU
CacheNode lru = tail.previous;
remove(lru);
map.remove(lru.key);
```

Variations

```
// VARIATION 1: LFU — adds a frequency dimension
// Need: key→value, key→freq, freq→orderedSet<key>, and minFreq
// When a key is accessed: freq++, move from freqToKeys[oldFreq] to freqToKeys[newFreq]
// On eviction: remove first element from freqToKeys[minFreq]
// LinkedHashSet preserves insertion order → oldest at front for same frequency

// VARIATION 2: Insert Delete GetRandom — use array for O(1) random
// ArrayList stores values; HashMap stores val→index in array
// On remove: swap target with last element, update map, remove last
// On getRandom: return list.get(random.nextInt(list.size()))
```

LRU vs LFU vs RandomizedSet — Pattern Chooser

Signal	Design
"evict least recently used"	LRU: HashMap + doubly linked list, move to head on access

"evict least frequently used"	LFU: 3 HashMaps + LinkedHashSet + minFreq tracking
"O(1) insert/delete/random"	RandomizedSet: HashMap + ArrayList, swap-with-last on delete

Key Invariants

LRU: head.next = MRU, tail.previous = LRU
LFU: minFreq always points to the smallest existing frequency bucket
LinkedHashSet preserves insertion order → iterator().next() = LRU within bucket
RandomizedSet: list[map[val]] == val at all times (after swap, update the map for the swapped key)

Math `math.md`

Quick Reference Table

#	Name	Description	Algorithm	Variation from Template	Time	Space
7	Reverse Integer	Reverse the digits of a signed 32-bit integer	Pop digits via %10, push via *10; guard overflow before each push	Overflow guard: check result > Integer.MAX_VALUE/10 before pushing	O(log n)	O(1)
9	Palindrome Number	Is the integer a palindrome (reads same forwards/backwards)?	Reverse only the second half; compare halves	Half reverse: stop when reversed >= original	O(log n)	O(1)
50	Pow(x, n)	Compute x raised to the power n	Fast exponentiation by squaring; handle negative n	Binary exponentiation: result *= x when (n & 1) == 1	O(log n)	O(1)
67	Add Binary	Add two binary strings, return the binary sum	Add from right with carry (like #2 but on strings)	Char arithmetic: (a-'0') + (b-'0') + carry, base 2	O(n)	O(n)
168	Excel Sheet Column Title	Convert column number → Excel title (1→A, 28→AB)	Base-26 conversion but 1-indexed (A = 1)	1-indexed: n-- before each digit extraction	O(log n)	O(1)
171	Excel Sheet Column Number	Convert Excel title → column number (AB→28)	Base-26 parse: result = result*26 + (c-'A'+1)	1-indexed letters: +1 so A maps to 1 not 0	O(L)	O(1)
204	Count Primes	Count primes strictly less than n	Sieve of Eratosthenes; mark multiples non-prime	Start at i*i: smaller multiples already marked	O(n log log n)	O(n)
372	Super Pow	a^b mod 1337 where b is a huge array of digits	Process b digit by digit with modular exponentiation	Digit-by-digit: a^b = (a^(b/10))^10 * a^(b%10)	O(log n)	O(1)
1201	Ugly Number III	Find nth positive integer divisible by a, b, or c	Binary search on the answer + inclusion-exclusion with LCM	Inclusion-exclusion: count multiples ≤ mid via LCMs	O(log min)	O(1)

Canonical Templates

```
// MENTAL MODEL: number = sequence of digits in some base; iterate by repeatedly
//                  peeling the last digit (% base) and shifting (/ base).
// WHEN: "reverse digits", "x^n", "base conversion", "count primes", "nth divisible-by".

// — DIGIT EXTRACTION LOOP — peel digits right-to-left
while (n != 0) {
    int digit = n % 10; // last digit
    n /= 10;           // drop last digit
    // ... use digit ...
}

// — FAST POWER (exponentiation by squaring) —
// WHY: x^n needs only O(log n) multiplications by squaring the base
//       and consuming one exponent bit per step.
// x^13 = x^8 * x^4 * x^1 (13 = 1101 in binary)
double fastPow(double x, long n) {
    double result = 1;
    while (n > 0) {
        if ((n & 1) == 1) result *= x; // bit set → include this power of x
        x *= x;                      // square the base for the next bit
        n >>= 1;                      // consume one exponent bit
    }
    return result;
}
```

```
// — BASE CONVERSION — generic base b, 0-indexed
// number → digits: peel with % b, shift with / b, collect, then reverse
StringBuilder builder = new StringBuilder();
while (num > 0) {
    builder.append(num % b);
    num /= b;
}
builder.reverse();
// digits → number: Horner's rule
int value = 0;
for (int digit : digits) value = value * b + digit;

// — SIEVE OF ERATOSTHENES — mark composites up to n
// WHY start at i*i: any multiple k*i with k < i was already marked by k.
boolean[] composite = new boolean[n];
for (int i = 2; i * i < n; i++) {
    if (composite[i]) continue;
    for (int j = i * i; j < n; j += i) // start at i*i, step by i
        composite[j] = true;
}
```

Math Cheat Sheet

```
n % 10           // last decimal digit
n /= 10          // drop last digit
result * 10 + digit // push a digit (build number left-to-right)
(n & 1) == 1      // odd
(n & 1) == 0      // even
n >>= 1          // halve (consume one bit)
x *= x           // square base in fast power
c - '0'          // char → digit value
c - 'A' + 1       // Excel letter → 1-indexed value
(char)( 'A' + d)  // 0..25 → 'A'..'Z'
i * i            // sieve marking start point
x / gcd(x,y) * y  // LCM without overflow (divide first)
```

Pattern Chooser

Signal	Technique	Time
"reverse the digits of a number"	Digit pop (%10) / push (*10) loop	O(log n)
"is the number a palindrome?"	Reverse only half, compare halves	O(log n)
"compute x^n / a^b efficiently"	Exponentiation by squaring (binary exp)	O(log n)
"x^huge mod m" / exponent given as array	Digit-by-digit modular fast power	O(log n)
"add two big numbers as strings"	Right-to-left add with carry	O(n)
"convert column number ↔ Excel title"	Base-26 conversion, 1-indexed	O(log n) / O(L)
"convert between number and base-b string"	Horner (parse) / peel-and-reverse (build)	O(L)
"count primes below n"	Sieve of Eratosthenes, mark from i*i	O(n log log n)
"nth number divisible by a/b/c"	Binary search on answer + inclusion-exclusion (LCM)	O(log range)
"check if n is power of 2/4"	Bit trick n & (n-1) == 0 (see bit_manipulation)	O(1)